



ຫົວບົດໂຄງການຈົບຊັ້ນລະດັບປະລິນຍາຕີວິສະວະກຳສາດ
ສາຂາ ວິສະວະກຳຄອມພິວເຕີ

ລະບົບກວດຈັບແລະຕອບສະໜອງໄພຄຸກຄາມໄຊເບີດ້ວຍການ
ໃຊ້ປັນຍາປະດິດ

**Artificial Intelligence Powered Cyber Threat Detection
And Response System**

ທ້າວ ພູນຈະເລີນໄຊ ສຸດທິວົງ

ທ້າວ ສຸພາ ຫຼວງນາມ

ຄະນະວິສະວະກຳສາດ

ສົກສຶກສາ 2025-2026

ຫົວບົດໂຄງການຈົບຊັ້ນລະດັບປະລິນຍາຕີວິສະວະກຳສາດ

ສາຂາ ວິສະວະກຳຄອມພິວເຕີ

ລະບົບກວດຈັບແລະຕອບສະໜອງໄພຄຸກຄາມໄຊເບີດ້ວຍການ
ໃຊ້ປັນຍາປະດິດ

ນັກສຶກສາ:

ທ້າວ ພູນຈະເລີນໄຊ ສຸດທິວົງ

ທ້າວ ສຸພາ ຫຼວງນາມ

ອາຈານຜູ້ນຳພາ:

ຮສ ແສງລັດສະໝີ ຈັນທະມີນາວົງ

ຄະນະວິສະວະກຳສາດ

ພາວີຊາ ວິສະວະກຳຄອມພິວເຕີ ແລະ ເຕັກໂນໂລຊີຂໍ້ມູນຂ່າວສານ

ສົກສຶກສາ 2025-2026

**Proposal Project of Bachelor Degree of Engineering
Department of computer engineering**

**Artificial Intelligence Powered Cyber Threat Detection
And Response System**

Students:

Mr. Phounjalearnzai SOUTHIVONG

Mr. Soupha LUANGNAM

Advisor:

Assoc. Port. Senglathsamy CHANTHAMYNAVONG

**Faculty of Engineering
Department of Computer Engineering and Information Technology
Academic Year 2025-2026**

ສາລະບານ

ບົດທີ 1 ບົດນຳສະເໜີ	3
1.1 ຄວາມເປັນມາ	3
1.2 ຈຸດປະສົງຂອງການສຶກສາ	4
1.3 ຂອບເຂດຂອງການສຶກສາ.....	4
1.4 ປະໂຫຍດທີ່ຄາດວ່າຈະໄດ້ຮັບ	5
1.4.1 ປະໂຫຍດທາງດ້ານວິຊາການ	5
1.4.2 ປະໂຫຍດນັກສຶກສາ	6
1.4.3 ປະໂຫຍດຕໍ່ອົງກອນ	6
1.4.4 ປະໂຫຍດຕໍ່ສັງຄົມ.....	6
1.5 ແຜນການດຳເນີນງານ	8
ບົດທີ 2 ທິດສະດີພື້ນຖານທີ່ກ່ຽວຂ້ອງ	9
2.1 ພາບລວມຄວາມປອດໄພທາງໄຊເບີ (Cybersecurity Overview).....	9
2.1.1 ຄວາມໝາຍ ແລະ ຄວາມສຳຄັນ.....	9
2.1.2 ຄວາມປອດໄພການຄຸກຄາມທາງໄຊເບີ (Types of Cyber Threats).....	9
2.1.3 ລະບົບກວດຈັບ ແລະ ປ້ອງກັນການບຸກລຸກ (IDS/IPS)	10
2.2 ລະບົບ SIEM (Security Information and Event Management)	10
2.2.1 ຄວາມໝາຍຂອງ SIEM.....	10
2.2.2 ຄວາມສຳຄັນຂອງລະບົບ SIEM.....	12
2.2.3 ພາລະບົດບາດ	13
2.2.4 ຮູບແບບ Log (Log Formats).....	13
2.2.5 Compliance Gate ແລະ ມາດຕະຖານສາກົນ	14

2.3	ປັນຍາປະດິດ ແລະ Machine Learning ສໍາລັບຄວາມປອດໄພ.....	14
2.3.1	Machine Learning (ML)	14
2.3.2	Deep Learning ສໍາລັບ Log Analysis	14
2.3.3	NLP ສໍາລັບ Log Analysis	15
2.3.4	ຕົວຊີ້ວັດການປະເມີນ (Evaluation Metrics).....	15
2.3.5	Dataset ທີ່ໃຊ້ໃນການຄົ້ນຄວ້າ.....	16
2.4	ເຄື່ອງມືແລະເຕັກໂນໂລຊີທີ່ໃຊ້.....	16
2.4.1	ແນະນຳ VIM	16
2.4.2	ແນະນຳ Zed Editor	17
2.4.3	ແນະນຳ Rust.....	17
2.4.4	ແນະນຳ Python.....	18
2.4.5	ແນະນຳ Pytorch.....	18
2.4.6	ແນະນຳ Docker	19
2.4.7	ແນະນຳ Fast API	19
2.4.8	ແນະນຳ Tokio + Axum	19
2.4.9	ແນະນຳ NIX	20
2.4.10	ແນະນຳ Google Cola.....	20
2.4.11	ແນະນຳ Kaggle.....	21
2.4.12	ແນະນຳ Hugging Face.....	21
2.4.13	ແນະນຳ Apache Kafka	21
ບົດທີ 3	ວິເຄາະ ແລະ ອອກແບບ	23
3.1	ວິເຄາະ	23
3.1.1	ການເຮັດວຽກຂອງລະບົບປັດຈຸບັນ	23

3.1.2 ບັນຫາ ແລະ ສາເຫດ	24
3.1.3 ແນວທາງແກ້ໄຂ	25
3.1.4 ຄວາມຕ້ອງການຂອງລະບົບໃໝ່	26
3.2 ການອອກແບບ	27
3.2.1 ການເຮັດວຽກຂອງລະບົບໃໝ່	27
3.2.2 ຊັບພະຍາກອນທີ່ນຳໃຊ້ພັດທະນາ	29
3.3 ການອອກແບບການທົດລອງ	30
3.3.1 ສະຖານະການທົດລອງ (Experiment Scenarios)	30
3.3.2 ຂໍ້ມູນ Dataset ທີ່ໃຊ້ໃນການທົດລອງ	31
3.3.3 Metrics ການປະເມີນ ແລະ ເກນຕັດສິນ	32
3.3.4 ຂັ້ນຕອນການທົດລອງ	32
3.3.5 ວິທີການພິສູດຜົນ (Proof Method)	33
3.4 ຂັ້ນຕອນການພັດທະນາລະບົບ	33
3.4.1 ການກະກຽມ ແລະ ການເກັບກຳ Dataset	33
3.4.2 ການພັດທະນາ AI/ML Models	33
3.4.3 ການນຳໃຊ້ Hugging Face ເຂົ້າໃນ Project	34
3.4.4 ການພັດທະນາ Backend System	36
3.4.5 ການລວມລະບົບ (System Integration)	37
3.4.6 ການທົດສອບ ແລະ ການປະເມີນ	37
3.5 ການຈັດການຄຸນນະພາບ	37
3.5.1 ການຮັບປະກັນຄຸນນະພາບ AI/ML Models	37
3.5.2 ການທົດສອບລະບົບ (System Testing)	38
3.5.3 ການປະເມີນປຽບທຽບ (Comparative Evaluation)	38

3.6	ການບໍລິຫານຈັດການຄວາມສູງ.....	39
-----	------------------------------	----

ສາລະບານຮູບພາບ

ຮູບພາບ 2.1 ສະແດງຮູບພາບລະບົບ SIEM	12
ຮູບພາບ 2.2 ສະແດງຮູບພາບ VIM	16
ຮູບພາບ 2.3 ສະແດງຮູບພາບ Zed Editor.....	17
ຮູບພາບ 2.4 ສະແດງຮູບພາບ Rust	17
ຮູບພາບ 2.5 ສະແດງຮູບພາບ Python.....	18
ຮູບພາບ 2.6 ສະແດງຮູບພາບ Pytorch.....	18
ຮູບພາບ 2.7 ສະແດງຮູບພາບ Docker.....	19
ຮູບພາບ 2.8 ສະແດງຮູບພາບ Fast API	19
ຮູບພາບ 2.9 ສະແດງຮູບພາບ Tokio + Axu.....	19
ຮູບພາບ 2.10 ສະແດງຮູບພາບ NIX.....	20
ຮູບພາບ 2.11 ສະແດງຮູບພາບ Google Cola	20
ຮູບພາບ 2.12 ‘ແນະນຳ Kaggle	21
ຮູບພາບ 2.13 ແນະນຳ Hugging Face	21
ຮູບພາບ 2.14 ແນະນຳ Apache Kafka.....	21
ຮູບພາບ 3.1 ສະແດງຮູບພາບການເຮັດວຽກລະບົບປັດຈຸບັນ	24
ຮູບພາບ 3.2 ສະແດງຮູບພາບການເຮັດວຽກຂອງລະບົບໃໝ່.....	27
ຮູບພາບ 3.3 ການຈຳແນກ Log (Log Classification & 4-Tier Scoring).....	29
ຮູບພາບ 3.4 ຕົວຢ່າງ code(Tier2).....	35
ຮູບພາບ 3.5 ຕົວຢ່າງ code(Tier3).....	35
ຮູບພາບ 3.6 ຂັ້ນຕອນການພັດທະນາ AL/ML	37
ຮູບພາບ 3.7 ການປຸງປຸງ Scenario A vs Scenario B	38

ສາລະບານຕາຕະລາງ

ຕາຕະລາງ 2.1 ພາລະບົດບາດ	13
ຕາຕະລາງ 2.2 ຕາຕະລາງໂຕຊີ້ວັດ.....	15
ຕາຕະລາງ 2.3 Dataset ທີ່ໃຊ້ໃນການຄົ້ນຄວ້າ.....	16
ຕາຕະລາງ 3.1 ສະແດງບັນຫາ ແລະ ສາເຫດ	24
ຕາຕະລາງ 3.2 ສະຖານະການທົດລອງ.....	31
ຕາຕະລາງ 3.3 ຂໍ້ມູນ Dataset	31
ຕາຕະລາງ 3.4 ການປະເມີນ ແລະ ການຕັດສິນ.....	32
ຕາຕະລາງ 3.5 Hugging Face Hub.....	34
ຕາຕະລາງ 3.6 ເຫດຜົນຂອງການນຳໃຊ້ Hugging Face	36
ຕາຕະລາງ 3.7 ການຮັບປະກັນຄຸນນະພາບ AL/ML	37
ຕາຕະລາງ 3.8 ການຈັດການຄວາມສົ່ງ.....	39

ບົດທີ 1

ບົດນຳສະເໜີ

1.1 ຄວາມເປັນມາ

ປັດຈຸບັນເຕັກໂນໂລຊີດ້ານຄອມພິວເຕີ ແລະ ເຄືອຂ່າຍອິນເຕີເນັດໄດ້ກາຍເປັນໂຄງສ້າງພື້ນຖານທີ່ຂາດບໍ່ໄດ້ສໍາລັບອົງກອນທຸກຂະໜາດ ໄລ່ຕັ້ງແຕ່ທ້ອງຖານລັດຖະກອນ, ສະຖາບັນການເງິນ, ມະຫາວິທະຍາໄລ ໄປຈົນເຖິງວິສາຫະກິດເອກະຊົນ. ສ່ວນຄຽງກັນນັ້ນ ປະລິມານຂໍ້ມູນບັນທຶກ (Log) ທີ່ສ້າງຂຶ້ນຈາກອຸປະກອນ IT ເຊັ່ນ Firewall, Server, Endpoint ແລະ Cloud Platform ກໍເພີ່ມຂຶ້ນຢ່າງໄວວາ.

ເພື່ອຈັດການກັບ Logs ເຫຼົ່ານີ້ອົງກອນຈຳນວນຫຼາຍໄດ້ນຳໃຊ້ລະບົບ Security Information and Event Management (SIEM) ເຊິ່ງເຮັດໜ້າທີ່ລວບລວມ, ວິເຄາະ ແລະ ແຈ້ງເຕືອນເຫດການດ້ານຄວາມປອດໄພ. ຢ່າງໃດກໍຕາມ SIEM ແບບດັ້ງເດີມ ທີ່ອາໄສ Correlation Rules ແບບ Static ຍັງປະສົບບັນຫາສໍາຄັນ 5 ປະການ ຄື:

- 1) ອັດຕາ False Positive ສູງ ເຮັດໃຫ້ທີມ SOC ເສຍເວລາກວດສອບ Alert ທີ່ບໍ່ແມ່ນໄພຄຸກຄາມຈິງ (Alert Fatigue).
- 2) ບໍ່ສາມາດກວດຈັບໄພຄຸກຄາມໃໝ່ທີ່ຍັງບໍ່ມີໃນຖານ Signature ໄດ້ ເຊັ່ນ Zero-day Attack ແລະ APT.
- 3) ເວລາຕອບສະໜອງຊ້າ ເນື່ອງຈາກຂຶ້ນກັບການວິເຄາະດ້ວຍຄົນ ຊຶ່ງອາດໃຊ້ເວລາຫຼາຍຊົ່ວໂມງ ຫຼື ຫຼາຍວັນ.
- 4) ຕົ້ນທຶນດໍາເນີນງານສູງ ທັງຄ່າ License ທີ່ຄິດໄລ່ຕາມ Events Per Second, ຄ່າ Storage ແລະ ຄ່າແຮງງານ SOC.
- 5) ຂາດແຄນບຸກຄະລາກອນຊ່ຽວຊານດ້ານ Cybersecurity ໃນ ສປປ ລາວ ໂດຍສະເພາະຜູ້ທີ່ມີປະສົບການດ້ານ Incident Response ບັນຫາຂ້າງເທິງນີ້ ສົ່ງຜົນໂດຍກົງຕໍ່ຄວາມປອດໄພຂອງຂໍ້ມູນໃນອົງກອນຂະໜາດໃຫຍ່ທີ່ດໍາເນີນງານ ໃນ ສປປ ລາວ ເຊັ່ນ: Lao Telecom (LTC), ທະນາຄານການຄ້າຕ່າງປະເທດລາວ (BCEL), ໄຟຟ້າລາວ (EDL) ແລະ ກະຊວງຕ່າງໆ ທີ່ເກັບຮັກສາຂໍ້ມູນລະອຽດອ່ອນ ແລະ ມີການເຊື່ອມຕໍ່ ເຄືອຂ່າຍຢ່າງກວ້າງຂວາງ.

ດ້ວຍເຫດດັ່ງນີ້ ການສຶກສາຄັ້ງນີ້ຈຶ່ງໄດ້ສະເໜີ AI-Powered Log Filtering Layer ທີ່ເຮັດວຽກຄຽງຄູ່ກັບ SIEM Platform ເດີມ ໂດຍນຳໃຊ້ Machine Learning (ML) ແລະ ເຕັກນິກ Natural Language Processing (NLP) ກວດຈັບໄພຄຸກຄາມແບບ Real-time ຫຼຸດ False Positive ແລະ ຕອບໂຕ້ອັດຕະໂນມັດ. ການສຶກສານີ້ຈະປຽບທຽບ 2 ສະຖານະການ ຄື:

- ສະຖານະການ A (Baseline): ລະບົບ SIEM ດັ້ງເດີມ ໂດຍບໍ່ມີ AI ຮັບ Logs ທັງ ໝົດໂດຍກົງ.

- ສະຖານະການ B (Proposed): ລະບົບ SIEM + AI Filtering Layer ຄັດກອງ Logs ດ້ວຍ AI ກ່ອນສົ່ງເຂົ້າ SIEM.

ຜົນທີ່ໄດ້ຈາກ Metrics ໄດ້ແກ່: Detection Rate, False Positive Rate, Log Reduction Rate, CPU/Memory/Storage ແລະ ຄ່າໃຊ້ຈ່າຍ ຊຶ່ງຈະສະທ້ອນໃຫ້ເຫັນ ວ່າ AI ຊ່ວຍໃຫ້ລະບົບ SIEM ທຳງານໄດ້ດີຂຶ້ນ ແລະ ປະທັບຄ່າໃຊ້ຈ່າຍຢ່າງຊັດເຈນ.

1.2 ຈຸດປະສົງຂອງການສຶກສາ

ຈຸດປະສົງຫຼັກຂອງການຄົ້ນຄວ້ານີ້ ມີ 3 ຂໍ້ ດັ່ງຕໍ່ໄປນີ້:

1) ເພື່ອພັດທະນາ AI Log Filtering Layer ທີ່ສາມາດຄັດກອງ ແລະ ຈຳແນກ Logs ທີ່ສຳຄັນ ກ່ອນສົ່ງເຂົ້າລະບົບ SIEM ຫຼຸດ EPS (Events Per Second) ທີ່ SIEM ຕ້ອງ ປະມວນຜົນ.

2) ເພື່ອພັດທະນາ ແລະ ຝຶກ AI/ML Model ສຳລັບກວດຈັບໄພຄຸກຄາມທາງໄຊເບີ (Threat Detection) ທີ່ສາມາດກວດຈັບທັງ Known Threats ຜ່ານ Sigma Rules ແລະ Unknown Threats ຜ່ານ Anomaly Detection.

3) ເພື່ອປຸງບາງ ແລະ ປະເມີນປະສິດທິພາບຂອງລະບົບທີ່ພັດທະນາຂຶ້ນ (Scenario B) ທຽບກັບ SIEM ດັ້ງເດີມ (Scenario A) ໂດຍໃຊ້ Metrics ມາດຕະຖານ ໄດ້ແກ່: Detection Rate, False Positive Rate ແລະ Log Reduction Rate.

1.3 ຂອບເຂດຂອງການສຶກສາ

ການສຶກສາມີຂອບເຂດດັ່ງນີ້:

1) ຂອບເຂດດ້ານຂໍ້ມູນ (Data Scope):

- ນຳໃຊ້ Logs ຈາກອຸປະກອນຄວາມປອດໄພ ແລະ IT ເຊັ່ນ: Firewall, IDS/IPS, Server, Endpoint ແລະ Application.

- ຂໍ້ມູນ Logs ເປັນ Benchmark Dataset ສາກົນໄດ້ແກ່: CICIDS2017, NSL-KDD ແລະ Log Hub.

2) ຂອບເຂດດ້ານເຕັກໂນໂລຊີ (Technology Scope):

- AI/ML ໄດ້ແກ່: Supervised Learning (XGBoost, LightGBM) ແລະ Unsupervised Learning (Isolation Forest, Autoencoder) ແລະ NLP (SecureBERT 2.0, TF-IDF).

- Backend ພັດທະນາດ້ວຍ Python (FastAPI) ຫຼື Rust (Tokio + Axum) ຕິດຕັ້ງໃນ Docker Container.

3) ຂອບເຂດດ້ານການປຸງບາງ ແລະ ປະເມີນຜົນ (Evaluation Scope):

- ທົດລອງ 2 ສະຖານະການ: (A) SIEM ດັ່ງເດີມ ແລະ (B) SIEM + AI Filtering ໂດຍໃຊ້ Dataset ຊຸດດຽວກັນໃນ Environment ດຽວກັນ ໄລຍະ 72 ຊົ່ວໂມງ.

- Metrics ຄຸນນະພາບ: Detection Rate, False Positive Rate, Log Reduction Rate, Precision, F1-Score.

- Metrics ຊັບພະຍາກອນ: CPU, Memory, Storage, Log Indexing Time ແລະ ຕົ້ນທຶນ (License, Storage, Man-hour).

- ໃນກໍລະນີຂອງການສຶກສານີ້ບໍ່ລວມການຕິດຕັ້ງໃຊ້ງານຈິງໃນອົງກອນ (Production Deployment) ພາຍໃນໄລຍະການຄົ້ນຄວ້ານີ້.

1.4 ປະໂຫຍດທີ່ຄາດວ່າຈະໄດ້ຮັບ

1.4.1 ປະໂຫຍດທາງດ້ານວິຊາການ

1) ຜົນງານການຄົ້ນຄວ້ານີ້ສະໜອງຄວາມຮູ້ໃໝ່ກ່ຽວກັບການນຳໃຊ້ AI/ML ໃນການກັ່ນຕອງ Logs ສຳ ລັບລະບົບ SIEM ຊຶ່ງຍັງເປັນຫົວຂໍ້ທີ່ມີການສຶກສາ.

2) ຄົ້ນຄວ້າຈຳກັດໃນ ສປປ ລາວ ໂດຍສະເພາະການປັບໃຊ້ເຕັກໂນໂລຊີກັບບໍລິບົດທ້ອງຖິ່ນ.

3) ພັດທະນາວິທີການປະເມີນປະສິດທິພາບຂອງລະບົບກວດຈັບໄພຄຸກຄາມທີ່ຊັດເຈນ ໂດຍອີງໃສ່ metrics ມາດຕະຖານສາກົນ (Detection Rate, False Positive Rate, Log Reduction Rate) ເຊິ່ງສາມາດນຳໃຊ້ເປັນກອບວິທີການສຳລັບການຄົ້ນຄວ້າໃນອະນາຄົດ.

4) ສ້າງຖານຂໍ້ມູນອ້າງອີງ (Benchmark Dataset) ສຳລັບການທົດສອບລະບົບຄວາມປອດໄພທາງໄຊເບີໃນສະພາບແວດລ້ອມຂອງ ສປປ ລາວ ເຊິ່ງຈະເປັນປະໂຫຍດຕໍ່ນັກຄົ້ນຄວ້າ ແລະ ສະຖາບັນການສຶກສາ.

5) ຜົນງານສາມາດເຜີຍແຜ່ເປັນບົດຄວາມວິຊາການ ຫຼື ນຳສະເໜີໃນກອງປະຊຸມວິຊາການ ທັງພາຍໃນ ແລະ ຕ່າງປະເທດ ຊ່ວຍຍົກລະດັບຊື່ສຽງດ້ານການຄົ້ນຄວ້າຂອງສະຖາບັນ.

1.4.2 ປະໂຫຍດນັກສຶກສາ

- 1) ໄດ້ຝຶກທັກສະດ້ານ Cybersecurity, AI/ML ແລະ Software Engineering ຢ່າງຄົບຖ້ວນ ຕັ້ງແຕ່ການກຽມ Dataset, ການສ້າງ Model ໄປຈົນເຖິງການ Deploy ດ້ວຍ Docker.
- 2) ໄດ້ຮຽນຮູ້ເຄື່ອງມືທີ່ອຸດສາຫະກຳໃຊ້ຈິງ (Industry Tools) ເຊັ່ນ IBM QRadar, Python, Rust, Kafka ແລະ Prometheus ຊ່ວຍເພີ່ມຄວາມສາມາດໃນການສະໝັກວຽກ.
- 3) ໄດ້ປະສົບການໃນການຄົ້ນຄວ້າ ແລະ ທົດລອງຢ່າງມີລະບົບຕາມຫຼັກວິທະຍາສາດ (Controlled Experiment) ຊຶ່ງເປັນທັກສະສໍາຄັນສໍາລັບການຮຽນຕໍ່ ຫຼື ວຽກງານຄົ້ນຄວ້າ.
- 4) ໄດ້ສ້າງ Portfolio ດ້ານ AI ແລະ Cybersecurity ທີ່ເປັນປະໂຫຍດຕໍ່ການສະໝັກທຶນ ຫຼື ຕໍາແໜ່ງງານທີ່ກ່ຽວຂ້ອງ.

1.4.3 ປະໂຫຍດຕໍ່ອົງກອນ

- 1) ເພີ່ມປະສິດທິພາບການກວດຈັບໄພຄຸກຄາມ ໂດຍສາມາດກວດພົບ Zero-day Attack ແລະ APT ທີ່ລະບົບດັ້ງເດີມທີ່ໃຊ້ Signature ບໍ່ສາມາດກວດຈັບໄດ້.
- 2) ຫຼຸດ False Positive ລົງໄດ້ຢ່າງຫຼວງຫຼາຍ, ຊ່ວຍໃຫ້ທີມ SOC ສາມາດສຸມໃສ່ໄພຄຸກຄາມທີ່ແທ້ຈິງ ແລະ ຫຼຸດ Alert Fatigue.
- 3) ຫຼຸດຄ່າໃຊ້ຈ່າຍດ້ານ License, Storage ແລະ ຄ່າແຮງງານ SOC ໂດຍ ROI ຄາດວ່າຈະຄືນທຶນໃນ 6–18 ເດືອນ ຂຶ້ນກັບຂະໜາດອົງກອນ.
- 4) ຮອງຮັບຄວາມຕ້ອງການດ້ານ Compliance ເຊັ່ນ: Cybersecurity Law of Lao PDR, ISO 27001, PCI-DSS ແລະ ອື່ນໆ, ຊ່ວຍຫຼີກລ່ຽງການຖືກລົງໂທດ ແລະ ຄວາມເສຍຫາຍທາງຊື່ສຽງ.

1.4.4 ປະໂຫຍດຕໍ່ສັງຄົມ

- 1) ຍົກລະດັບຄວາມສາມາດດ້ານໄຊເບີຂອງ ສປປ ລາວ ໃຫ້ທັນກັບມາດຕະຖານສາກົນ ແລະ ພ້ອມຮັບມືກັບໄພຄຸກຄາມໃນຍຸກດິຈິຕອລ.
- 2) ປ້ອງກັນໂຄງສ້າງພື້ນຖານຂອງຊາດ ເຊັ່ນ: ລະບົບພະລັງງານ (EDL), ລະບົບການເງິນ (BoL, BCEL) ແລະ ລະບົບໂທລະຄົມ (LTC, Unitel, ETL) ຈາກການໂຈມຕີທາງໄຊເບີ.
- 3) ສ້າງຖານພື້ນຖານສໍາລັບການພັດທະນາ Digital Economy ແລະ Smart Laos ໂດຍສ້າງຄວາມເຊື່ອໝັ້ນໃນຄວາມປອດໄພດ້ານດິຈິຕອລ.
- 4) ພັດທະນາບຸກຄະລາກອນທ້ອງຖິ່ນທີ່ມີຄວາມຊຸ່ວຊານດ້ານປັນຍາປະດິດ ແລະ Cybersecurity ເຊິ່ງຈະເປັນກໍາລັງສໍາຄັນໃນການພັດທະນາເສດຖະກິດດິຈິຕອລຂອງ ສປປ ລາວ.

1.5 ແຜນການດຳເນີນງານ

ສະແດງລາຍລະອຽດຂອງການສຶກສາຄັ້ງນີ້ໄດ້ວາງແຜນໄວ້ໃນໄລຍະການດຳເນີນງານໃນສົກຮຽນ 2025–2026.

ຕາຕະລາງທີ 1.1 ຕາຕະລາງໄລຍະເວລາດຳເນີນງານ

ລ/ດ	ປະຕິບັດ	ຊ່ວງໄລຍະເວລາ Proposal																ຊ່ວງໄລຍະເວລາ Project															
		ເດືອນ 1				ເດືອນ 2				ເດືອນ 3				ເດືອນ 4				ເດືອນ 5				ເດືອນ 6				ເດືອນ 7							
		w1	w2	w3	w4	w1	2w	w3	w4	w1	w2	w3	w4	w1	w2	w3	w4	w1	w2	w3	w4	w1	w2	w3	w4	w1	w2	w3	w4				
1	ເລືອກອາຈານທີ່ປຶກສາ																																
2	ເລືອກຫົວບົດ																																
3	ວິເຄາະບັນຫາ ແລະ ສາເຫດຂອງລະບົບໃໝ່ປັດຈຸບັນ																																
4	ກຳນົດຄວາມຕ້ອງການຂອງລະບົບໃໝ່																																
5	ປັບປຸງ ແລະ ແກ້ໄຂເອກະສານ																																
6	ອອກແບບລະບົບໃໝ່																																
7	ການສ້າງ Architecture																																
8	ກຳນົດຂອບເຂດຂອງລະບົບ																																
ປ້ອງກັນ Proposal																																	
9																																	
10	ການຈັດລຽນປະເພດຂອງ Log																																
11	ການຕິດຕັ້ງ ແລະ ທົດທະນາໄປແກຣມ																																
12	ການສ້າງ API Gateway																																
13	ການສ້າງ Train Model																																
14	ການປັບປຸງປະສິດທິພາບຂອງລະບົບ																																
15	ການເຊື່ອມ API ເຂົ້າເຄື່ອງມື SIEM ແລະ ທົດສອບລະບົບ																																
ປ້ອງກັນບົດໂຄງການຈົບຊັ້ນ																																	

ບົດທີ 2

ທິດສະດີພື້ນຖານທີ່ກ່ຽວຂ້ອງ

2.1 ພາບລວມຄວາມປອດໄພທາງໄຊເບີ (Cybersecurity Overview)

2.1.1 ຄວາມໝາຍ ແລະ ຄວາມສໍາຄັນ

ຄວາມປອດໄພທາງໄຊເບີ (Cybersecurity) ໝາຍເຖິງຊຸດຂອງເຕັກໂນໂລຊີ, ຂະບວນການ ແລະ ການປະຕິບັດທີ່ຖືກອອກແບບເພື່ອປົກປ້ອງລະບົບຄອມພິວເຕີ, ເຄືອຂ່າຍ ແລະ ຂໍ້ມູນຈາກການໂຈມຕີ, ຄວາມເສຍຫາຍ ຫຼື ການເຂົ້າເຖິງໂດຍບໍ່ໄດ້ຮັບອະນຸຍາດ. ໃນຍຸກດິຈິຕອລ, ອົງກອນທຸກຂະໜາດຕ້ອງການລະບົບຄວາມປອດໄພທາງໄຊເບີທີ່ເຂັ້ມງວດ ເນື່ອງຈາກໄພຄຸກຄາມທາງໄຊເບີໄດ້ພັດທະນາຢ່າງຕໍ່ເນື່ອງ.

2.1.2 ຄວາມປອດໄພການຄຸກຄາມທາງໄຊເບີ (Types of Cyber Threats)

ໄພຄຸກຄາມທາງໄຊເບີ (Cyber Threats) ໝາຍເຖິງການກະທຳ ຫຼື ເຫດການທີ່ເປັນໄປໄດ້ຊຶ່ງອາດຈະທຳໃຫ້ເສຍຫາຍຕໍ່ຄວາມລັບ (Confidentiality), ຄວາມຄົງທົນ (Integrity) ຫຼື ຄວາມພ້ອມໃຊ້ງານ (Availability) ຂອງຂໍ້ມູນ ຫຼື ລະບົບ. ປະເພດຫຼັກທີ່ກ່ຽວຂ້ອງກັບລະບົບ SIEM ໃນການສຶກສານີ້ ມີດັ່ງຕໍ່ໄປ:

1) Malware (ມາລແວ): ໂຄດທີ່ເປັນອັນຕະລາຍຊຶ່ງຖືກອອກແບບມາເພື່ອລົບກວນ, ເສຍຫາຍ ຫຼື ເຂົ້າເຖິງລະບົບໂດຍບໍ່ໄດ້ຮັບອະນຸຍາດ. Malware ຖືກຈຳແນກອອກເປັນຫຼາຍປະເພດ:

- Virus: ຕິດຊ້ອນໃນໄຟລ໌ທົ່ວໄປ ແລ້ວຈຳລອງຕົວເອງເມື່ອໄຟລ໌ຖືກເຮັດວຽກ.
- Worm: ແຜ່ຂະຫຍາຍໂດຍອິດຕະໂນມັດຜ່ານເຄືອຂ່າຍ ໂດຍບໍ່ຕ້ອງອາຜູ້ໃຊ້.
- Trojan: ປອມເປັນໂປຼແກຼມທີ່ໜ້າໄວ້ໃຈ ແຕ່ຊ່ອນຄວາມສາມາດທີ່ເປັນ.

ອັນຕະລາຍ.

- Ransomware: ເຂົ້າລະຫັດໄຟລ໌ ແລ້ວຮຽກຮ້ອງຄ່າໄຖ່ເພື່ອແລກຄືນ.
- Rootkit: ຊ່ອນຕົວຢູ່ໃນ OS ເພື່ອໃຫ້ Attacker ເຂົ້າຄວບຄຸມ.
- Spyware / Keylogger: ລັກຂໍ້ມູນ ເຊັ່ນ: ລະຫັດຜ່ານ ແລະ ຂໍ້ມູນທາງການເງິນ.

2) Advanced Persistent Threat (APT): ການໂຈມຕີທີ່ຊັບຊ້ອນ, ຍາວນານ ແລະ ມີເປົ້າໝາຍຊັດເຈນ ໂດຍຜູ້ໂຈມຕີທີ່ມີທຶນຮອນ ແລະ ທັກສະສູງ. APT ມີຄຸນລັກສະນະດັ່ງວ່າ:

- ໄລຍະ Reconnaissance: ສຶກສາເປົ້າໝາຍຢ່າງລະອຽດກ່ອນໂຈມຕີ
- ໄລຍະ Lateral Movement: ເຄື່ອນທີ່ຂ້າມລະຫວ່າງລະບົບຢ່າງງ່າຍດາຍ
- ໄລຍະ Exfiltration: ສົ່ງຂໍ້ມູນລັບອອກໄປໂດຍຫຼີກລ່ຽງການກວດຈັບ

3) Zero-day Attack: ການໂຈມຕີທີ່ໃຊ້ຊ່ອງໂຫວ່ (Vulnerability) ທີ່ຜູ້ຂາຍຊອບແວ ຫຼື ຊຸມຊົນຄວາມປອດໄພຍັງບໍ່ຮູ້ຈັກ ຊຶ່ງໝາຍຄວາມວ່າຍັງບໍ່ມີ Patch ເພື່ອແກ້ໄຂ. ລະບົບ SIEM ແບບດັ້ງເດີມທີ່ໃຊ້ Signature-based Detection ຈະບໍ່ສາມາດກວດຈັບໄດ້ ຕ້ອງການ Anomaly Detection ດ້ວຍ AI/ML ເທົ່ານັ້ນ.

4) DDoS (Distributed Denial of Service): ການໂຈມຕີທີ່ສົ່ງ Traffic ຈຳນວນຫຼວງຫຼາຍໄປຍັງ Server ຫຼື ເຄືອຂ່າຍ ເພື່ອທຳໃຫ້ການໃຫ້ບໍລິການຢຸດ ຫຼື ຊ້າລົງ. ອາດມີໃຊ້ Botnet ຫຼາຍສິບພັນເຄື່ອງໃນການໂຈມຕີພ້ອມກັນ.

5) Insider Threat (ໄພຄຸກຄາມພາຍໃນ): ໄພຈາກພະນັກງານ ຫຼື ຜູ້ທີ່ມີສິດ Access ທີ່ຈົງໃຈ ຫຼື ບໍ່ຈົງໃຈເຮັດໃຫ້ຂໍ້ມູນຮົ່ວໄຫຼ ຫຼື ລະບົບຖືກທຳລາຍ. ຍາກທີ່ຈະກວດຈັບດ້ວຍ Perimeter Security ຢ່າງດຽວ ຕ້ອງໃຊ້ User Behavior Analytics (UBA).

2.1.3 ລະບົບກວດຈັບ ແລະ ປ້ອງກັນການບຸກລຸກ (IDS/IPS)

Intrusion Detection System (IDS) ແລະ Intrusion Prevention System (IPS) ແມ່ນລະບົບທີ່ຕິດຕາມ Network Traffic ແລະ ກວດຈັບ ຫຼື ສະກັດກັ້ນກິດຈະກຳທີ່ໜ້າສົງໄສ. ຄວາມແຕກຕ່າງຫຼັກ:

- IDS (Detection Only): ຕິດຕາມ Traffic ແລ້ວ Alert ເທົ່ານັ້ນ ບໍ່ສາມາດ Block ໄດ້ຕື່ງ.

- IPS (Detection + Prevention): ສາມາດ Block Traffic ທີ່ໜ້າສົງໄສໄດ້ທັນທີ ວິທີການກວດຈັບຂອງ IDS/IPS ມີ 2 ປະເພດຫຼັກ:

- Signature-based Detection: ປຸງປຸງ Traffic ກັບຖານຂໍ້ມູນ Attack Signatures — ຖືກ False Positive ຕ່ຳ ແຕ່ຕ້ານ Zero-day ບໍ່ໄດ້
- Anomaly-based Detection: ຮຽນຮູ້ພຶດຕິກຳປົກກະຕິ ແລ້ວ Alert ເມື່ອພົບຄວາມຜິດປົກກະຕິ ສາມາດຕ້ານ Zero-day ໄດ້ແຕ່ False Positive ສູງກວ່າ.

2.2 ລະບົບ SIEM (Security Information and Event Management)

2.2.1 ຄວາມໝາຍຂອງ SIEM

ອົງປະກອບທີ່ສຳຄັນອັນໜຶ່ງຂອງຄວາມປອດໄພທາງຊເບີ Cybersecurity ທີ່ປະສິດທິພາບ ແມ່ນລະບົບ SIEM (Security Information and Event Management). ເຄື່ອງມືປະເພດນີ້ຈະເຮັດ

ໜ້າທີ່ເກັບກຳ, ຮວບຮວມ ແລະ ວິເຄາະຂໍ້ມູນຈຳນວນມະຫາສານຈາກແອັບພລິເຄຊັນ, ອຸປະກອນ, ເຊີເວີ ແລະ ຜູ້ໃຊ້ງານທົ່ວທັງອົງກອນແບບທັນການ (Real-time).

ດ້ວຍການຮວບຮວມຂໍ້ມູນທີ່ຫຼາກຫຼາຍເຫຼົ່ານີ້ເຂົ້າໄວ້ໃນແພລດຟອມດຽວທີ່ເປັນເອກະພາບ, SIEM ຊ່ວຍໃຫ້ເຫັນພາບລວມຂອງສະຖານະຄວາມປອດໄພຂອງອົງກອນຢ່າງຄົບຖ້ວນ, ເຊິ່ງເປັນການເພີ່ມຂີດຄວາມສາມາດໃຫ້ກັບ ສູນປະຕິບັດການດ້ານຄວາມປອດໄພ (SOC) ໃນການກວດຈັບ ແລະ ຕອບໂຕ້ກັບເຫດການດ້ານຄວາມປອດໄພໄດ້ຢ່າງວ່ອງໄວ ແລະ ມີປະສິດທິພາບ. SIEM ສາມາດຊ່ວຍອົງກອນທຸກຂະໜາດໄດ້ດັ່ງນີ້:

- 1) ສ້າງທັດສະນະວິໄສໃນການເບິ່ງເຫັນສະຖານະຄວາມປອດໄພ: ໂດຍການລວມສູນ ແລະ ວິເຄາະຂໍ້ມູນຈາກແຫຼ່ງຕ່າງໆ ທີ່ແຕກຕ່າງກັນ.
- 2) ກວດຈັບ ແລະ ລະບຸການບຸກລຸກ: ລວມເຖິງໄພຄຸກຄາມທີ່ອາດຈະເກີດຂຶ້ນແບບ Real-time ເພື່ອຫຼຸດຜ່ອນຄວາມສ່ຽງຈາກການຖືກໂຈມຕີ.
- 3) ສືບສວນ ແລະ ຄັດກອງເຫດການ: ຊ່ວຍໃຫ້ການຈັດການກັບເຫດການດ້ານຄວາມປອດໄພມີປະສິດທິພາບຫຼາຍຂຶ້ນ, ຫຼຸດຜ່ອນເວລາ ແລະ ຊັບພະຍາກອນທີ່ຕ້ອງໃຊ້ໃນການແກ້ໄຂບັນຫາ.
- 4) ປະຕິບັດຕາມລະບຽບການ: ຊ່ວຍໃຫ້ສອດຄ່ອງກັບມາດຕະຖານຄວາມປອດໄພ ແລະ ຂອບເຂດວຽກງານ (Framework) ຕາມກົດລະບຽບ ຫຼື ຂໍ້ກຳນົດສະເພາະຂອງຂະແໜງການ.

(ຮູບໄດ້ເອົາມາອ້າງອີງຈາກ Miller, 2026)



ຮູບພາບ 2.1 ສະແດງຮູບພາບລະບົບ SIEM

2.2.2 ຄວາມສໍາຄັນຂອງລະບົບ SIEM

ພາບຄໍານິຍາມຂອງ SIEM ພາຍໃນຄວາມປອດໄພທາງໄຊເບີແມ່ນໃຊ້ໄດ້ໂດຍສະເພາະກັບ ອົງກອນທີ່ດໍາເນີນງານໃນສະພາບແວດລ້ອມທີ່ມີກົດລະບຽບສູງ ຫຼື ຈັດການກັບຂໍ້ມູນທີ່ລະອຽດ ອ່ອນ. ອົງກອນເຫຼົ່ານີ້ມັກຈະຕ້ອງປະຕິບັດຕາມມາດຕະຖານການປະຕິບັດຕາມທີ່ເຂັ້ມງວດ, ແລະ SIEM ຊ່ວຍແກ້ໄຂຄວາມຕ້ອງການເຫຼົ່ານີ້ຜ່ານການຕິດຕາມກວດກາໃນເວລາຈິງ ແລະ ການກວດ ສອບລາຍລະອຽດຂອງເຫດການຄວາມປອດໄພ.

ນອກຈາກນັ້ນ, ຍ້ອນວ່າໄພຂົ່ມຂູ່ທາງໄຊເບີຍັງສືບຕໍ່ພັດທະນາ, ອົງກອນຕ່າງໆຕ້ອງປ່ຽນບູລິ ມະສິດຂອງເຂົາເຈົ້າຈາກການປ້ອງກັນການໂຈມຕີໄປສູ່ການກວດຈັບ ແລະ ການຕອບສະໜອງໃນ ເວລາຈິງ. ວິທີແກ້ໄຂ SIEM ໃຫ້ຈຸດຍືນດ້ານຄວາມປອດໄພທີ່ມີປະສິດທິພາບ, ຊ່ວຍໃຫ້ທີມງານ ດ້ານຄວາມປອດໄພສາມາດລະບຸຊ່ອງໂຫວ່, ວິເຄາະຂໍ້ມູນ ແລະ ຕອບສະໜອງຕໍ່ໄພຂົ່ມຂູ່ກ່ອນທີ່ ມັນຈະສາມາດກໍ່ໃຫ້ເກີດຄວາມເສຍຫາຍທີ່ບໍ່ສາມາດແກ້ໄຂໄດ້.

2.2.3 ພາລະບົດບາດ

SIEM ປະກອບດ້ວຍ 3 ໜ້າທີ່ຫຼັກທີ່ເຊື່ອມໂຍງກັນ:

ຕາຕະລາງ 2.1 ພາລະບົດບາດ

ອົງປະກອບ	ພາລະບົດບາດ	ຕົວຢ່າງ
Log Collection	ເກັບ Logs ຈາກທຸກ Source ດ້ວຍ Agent ຫຼື Syslog	Firewall, IDS, Server, App
Event Correlation	ຈັບຄູ່ Events ຈາກຫຼາຍ Source ເພື່ອຊອກຫາຮູບແບບໄພຄຸກຄາມ	Splunk Correlation Rules
Alerting & Dashboard	ສ້າງ Alert ແລະ ສະແດງ Dashboard ໃຫ້ SOC Team	Wazuh, Elastic SIEM
Log Storage & Search	ເກັບຮັກສາ Logs ໄລຍະຍາວ ແລະ ສາມາດ Query ໄດ້ໄວ	Elasticsearch, OpenSearch
Compliance Reporting	ສ້າງ Report ຕາມ Standard ເຊັ່ນ PCI-DSS, ISO 27001	Automated Report Generation

ບັນຫາຂອງ SIEM ແບບດັ້ງເດີມທີ່ການສຶກສານີ້ຕ້ອງການແກ້ໄຂ ຄືການຮັບ Logs ຈຳນວນ ຫຼວງຫຼາຍ ສົ່ງຜົນໃຫ້ Alert Fatigue ສູງ, False Positive ຫຼາຍ ແລະ ຄ່າ License ແພງ (ຄິດໄລ່ ຕາມ Events Per Second). AI Log Filtering Layer ທີ່ຈະພັດທະນາ ຈະເຮັດໜ້າທີ່ຄັດກອງ Logs ທີ່ສໍາຄັນກ່ອນສົ່ງເຂົ້າ SIEM.

2.2.4 ຮູບແບບ Log (Log Formats)

Log Format ທີ່ SIEM ຕ້ອງຈັດການ ມີຫຼາຍຮູບແບບ ຄື:

- Syslog (RFC 5424): ມາດຕະຖານ IETF ສໍາລັບ Linux/Unix — ປະກອບດ້ວຍ PRI (Facility+Severity), Header (Timestamp+Hostname) ແລະ MSG.
- JSON Log: ຮູບແບບ Key-Value ທີ່ parse ໆ່ງ່າຍ — ນິຍົມໃຊ້ໃນ Web Applications ແລະ Microservices.
- CEF (Common Event Format): ຮູບແບບ ArcSight ທີ່ SIEM ຫຼາຍ Vendor ຮອງຮັບ.
- LEEF (Log Event Extended Format): ຮູບແບບຂອງ IBM QRadar ສໍາລັບ Enterprise .
- CLF (Common Log Format): ຮູບແບບ Apache HTTP Server ເຊັ່ນ: 127.0.0.1 – [timestamp] "GET /index.html" 200 2326.

2.2.5 Compliance Gate ແລະ ມາດຕະຖານສາກົນ

ໜຶ່ງໃນຄຸນຄ່າຫຼັກຂອງລະບົບ (SIEM) ຄືການຊ່ວຍອົງກອນປະຕິບັດຕາມກົດໝາຍ ແລະ ມາດຕະຖານ (Compliance). ມາດຕະຖານທີ່ກ່ຽວຂ້ອງກັບລະບົບໃນ ສປປ ລາວ ໄດ້ແກ່:

- Cybersecurity Law of Lao PDR: ກົດໝາຍວ່າດ້ວຍຄວາມປອດໄພທາງໄຊເບີຂອງ ສປປ ລາວ.
- ISO/IEC 27001: ມາດຕະຖານການຄຸ້ມຄອງຄວາມປອດໄພຂໍ້ມູນ (ISMS) ສາກົນ.
- PCI-DSS: ສໍາລັບສະຖາບັນການເງິນທີ່ຮ່ວມງານລະຫວ່າງຊາດ — 12 ຂໍ້ກຳນົດຄອບຄຸມ Firewall ເຖິງ Security Policy.
- GDPR: ກົດລະບຽບ EU ສໍາລັບອົງກອນທີ່ຈັດການຂໍ້ມູນພົນລະເມືອງເອີຣົບ.
- ISO/IEC 27002:2022 (Section 8.15): ໃຫ້ Logging ເປັນ Control ທີ່ຈຳເປັນ.

2.3 ບັນຍາຍພຶດຕິ ແລະ Machine Learning ສໍາລັບຄວາມປອດໄພ

2.3.1 Machine Learning (ML)

Machine Learning (ML) ໝາຍເຖິງໂປຼແກຼມຄອມພິວເຕີທີ່ "ຮຽນຮູ້" ຈາກຂໍ້ມູນ ໂດຍ Performance ດີຂຶ້ນຕາມປະສົບການທີ່ເພີ່ມຂຶ້ນ ໂດຍອີງໃສ່ Statistical Learning Theory (Vapnik, 1998). ສໍາລັບ Log Filtering ໃນລະບົບ SIEM ການສຶກສານີ້ ML ຖືກນໍາໃຊ້ 3 ແນວທາງ: Supervised Learning: ໃຊ້ຂໍ້ມູນ Labeled (Normal vs. Malicious) ຝຶກ Model ຈຳແນກ Log. ເຕັກນິກທີ່ໃຊ້ໃນການສຶກສານີ້:

- XGBoost: Gradient Boosting ທີ່ໄວ ແລະ ຖືກຕ້ອງ ສໍາລັບ Tabular Log Data.
 - LightGBM: ເໝາະ ລະບົບ Log ຂະໜາດໃຫຍ່ ດ້ວຍ Memory ໜ້ອຍກວ່າ XGBoost
 - Random Forest: ທົນທານຕໍ່ Noise ໃນ Log Files
- Unsupervised Learning: ກວດຈັບ Anomaly ໂດຍບໍ່ຕ້ອງ Label — ສໍາຄັນສໍາລັບ Zero-day Detection:
- Isolation Forest: ກວດ Outlier ໃນ Log ທີ່ Behavior ຜິດປົກກະຕິ
 - Autoencoder: ຮຽນຮູ້ Reconstruction ຂອງ Normal Log — Log ທີ່ Reconstruct ຍາກ = Anomaly.

2.3.2 Deep Learning ສໍາລັບ Log Analysis

Deep Learning ໃຊ້ Neural Networks ຫຼາຍຊັ້ນ ວິເຄາະຮູບແບບທີ່ຊັບຊ້ອນໃນ Log. ປະເພດທີ່ກ່ຽວຂ້ອງໂດຍກົງ:

- LSTM (Long Short-Term Memory): ວິເຄາະ Log Sequence ຕາມລຳດັບ ເວລາ — ກວດຈັບ Slow-burn Attacks ທີ່ກ່ຽວຂ້ອງຫຼາຍ Events.
- Transformer / BERT: ໃຊ້ Attention Mechanism ວິເຄາະ Log ທັງໝົດພ້ອມ ກັນ — ດີຂຶ້ນກວ່າ LSTM ສຳລັບ Long Log Sequences.
- SecureBERT 2.0: BERT ທີ່ Fine-tune ສຳລັບ Cybersecurity ໂດຍສະເພາະ — ໃຊ້ Vocabulary ຂອງ Security ໂດຍທົ່ວໄປ.

2.3.3 NLP ສຳລັບ Log Analysis

Natural Language Processing (NLP) ຖືກນຳໃຊ້ໃນ Log Analysis ເນື່ອງຈາກ Log Messages ແມ່ນ Text ທີ່ຕ້ອງ Parse ຄວາມໝາຍ. ເຕັກນິກ NLP ທີ່ໃຊ້:

- TF-IDF (Term Frequency-Inverse Document Frequency): ສ້າງ Feature Vector ຈາກ Log Text ເພື່ອຈຳແນກ Normal vs. Attack Logs.
- Tokenization & Parsing: ແຍກ Log Entry ອອກເປັນ Token (IP, Timestamp, Action, Status) ກ່ອນ Feed ໃຫ້ Model.
- Word Embedding: ແປ Log Tokens ໃຫ້ເປັນ Vector ທີ່ Capture ຄວາມໝາຍ ເຊັ່ນ: word2vec, FastText.

2.3.4 ຕົວຊີ້ວັດການປະເມີນ (Evaluation Metrics)

ເພື່ອວັດປະສິດທິພາບຂອງ AI Log Filter ແລະ ປຸງບທຽບ SIEM ດັ້ງເດີມ vs SIEM+AI, ການສຶກສານີ້ໃຊ້ Metrics ອ້າງອີງຈາກ Confusion Matrix:

ຕາຕະລາງ 2.2 ຕາຕະລາງໂຕຊີ້ວັດ

Metric	ສູດຄຳນວນ	ຄວາມໝາຍ
Detection Rate (Recall)	$TP / (TP + FN)$	ອັດຕາການກວດຈັບ Incident ຈິງ ຕ້ອງການສູງ
False Positive Rate (FPR)	$FP / (FP + TN)$	ອັດຕາ Alert ທີ່ຜິດຕ້ອງການຕໍ່າ
Precision	$TP / (TP + FP)$	ຄວາມຖືກຕ້ອງຂອງ Positive Prediction
F1-Score	$2 \times (Precision \times Recall) / (Precision + Recall)$	ຄ່າ Balance ລະຫວ່າງ Precision ແລະ Recall

Log Reduction Rate	$(\text{Logs_before} - \text{Logs_after}) / \text{Logs_before} \times 100\%$	ອັດຕາ % Logs ທີ່ຄັດກອງໄດ້
--------------------	---	---------------------------

ໝາຍເຫດ: TP = True Positive (ກວດຈັບ Attack ໄດ້ຖືກ), FN = False Negative (ກວດ Attack ໄດ້ຜິດ), FP = False Positive (Alert ທີ່ຜິດ), TN = True Negative (ຖືກຕ້ອງວ່າ Normal).

2.3.5 Dataset ທີ່ໃຊ້ໃນການຄົ້ນຄວ້າ

Dataset ທີ່ຖືກເລືອກສໍາລັບ Train ແລະ Evaluate AI Model ໃນການສຶກສານີ້:

ຕາຕະລາງ 2.3 Dataset ທີ່ໃຊ້ໃນການຄົ້ນຄວ້າ

Dataset	ລາຍລະອຽດ	ການໃຊ້ງານ
CICIDS2017	Network Intrusion Dataset ຈາກ Univ. of New Brunswick — 15 ປະເພດ Attack	Train Supervised Model
NSL-KDD	ປັບປຸງຈາກ KDD Cup 99 — ຄວບຄຸມ 4 ໝວດ Attack: DoS, R2L, U2R, Probe	Benchmark ປຸງປຸງ
Log Hub	ຊຸດ System Logs ຈາກ Real-world Systems (HDFS, BGL, Windows, Linux)	Test Log Parsing & Anomaly
ຂໍ້ມູນ Anonymized ຈາກອົງກອນ	Logs ຈິ່ງທີ່ Anonymize ໂດຍ Partner Organizations	Validate ໃນ Real Context

2.4 ເຄື່ອງມືແລະເຕັກໂນໂລຊີທີ່ໃຊ້

ເຄື່ອງມື ແລະ ເຕັກໂນໂລຊີທີ່ນຳມາໃຊ້ໃນການອອກແບບມີຄື: Zed editor , Vim , Python ,Rust , Pytorch ແລະ ອື່ນໆ.

2.4.1 ແນະນຳ VIM



ຮູບພາບ 2.2 ສະແດງຮູບພາບ VIM

Vim ແມ່ນ Text Editor ທີ່ນິຍົມໃນກຸ່ມນັກພັດທະນາລະບົບເນື່ອງຈາກມີຄວາມໄວສູງ, ໃຊ້ຊັບພະຍາກອນນ້ອຍແລະ ສາມາດນຳໃຊ້ໄດ້ດີໃນສະພາບແວດລ້ອມ Server, ເນັ້ນໄປທາງ Coding ແລະ System Administration.

2.4.2 ແນະນຳ Zed Editor



ຮູບພາບ 2.3 ສະແດງຮູບພາບ Zed Editor

ເປັນ Zed Editor ແມ່ນເຄື່ອງມືສຳລັບຂຽນໂຄດ (Code Editor) ທີ່ທັນສະໄໝ ແລະ ມີຄວາມໄວສູງເພາະສຳລັບການພັດທະນາຊອບແວຂະໜາດໃຫຍ່ Zed Editor ເຫມາະສົມແກ່ developer ທີ່ຕ້ອງການ **Speed** ແລະ **Real-time collaboration**, ມີ UI ເຂົ້າໃຈງ່າຍ ແລະ performance ດີ. ສະເພາະໜ້າທີ່ໃຊ້ coding ໃນ modern programming languages ເຊັ່ນ Python, JavaScript.

2.4.3 ແນະນຳ Rust



ຮູບພາບ 2.4 ສະແດງຮູບພາບ Rust

Rust ແມ່ນພາສາການຂຽນໂປຣແກຣມທີ່ໃຫ້ຄວາມປອດໄພສູງ ແລະ ປະສິດທິພາບສູງໃນລະດັບລະບົບ, Rust ແມ່ນພາສາການຂຽນໂປຣແກຣມລະດັບລະບົບ (System Programming Language) ທີ່ຖືກອອກແບບເພື່ອໃຫ້ມີຄວາມປອດໄພສູງໂດຍສະເພາະໃນດ້ານ Memory Safety ແລະ Concurrency ຈຸດເດັ່ນທີ່ສຳຄັນຂອງ Rust ຄືລະບົບ Ownership, Borrowing ແລະ Lifetime ຊຶ່ງຊ່ວຍປ້ອງກັນບັນຫາ Memory Leak, Dangling Pointer ແລະ Data Race ໂດຍບໍ່ຈຳເປັນຕ້ອງ

ໃຊ້ Garbage Collector ,Rust ຖືກນຳໃຊ້ເພື່ອພັດທະນາ Backend ຂອງລະບົບແລະ ປະມວນຜົນເຫດການ (Events) ຈຳນວນຫຼາຍໃນເວລາຈິງເພື່ອໃຫ້ລະບົບມີຄວາມສະດວກ ແລະ ສາມາດຮອງຮັບການຂະຫຍາຍໃນອະນາຄົດ.

2.4.4 ແນະນຳ Python



ຮູບພາບ 2.5 ສະແດງຮູບພາບ Python

ໂປຣ Python ແມ່ນພາສາການຂຽນໂປຣແກຣມທີ່ເໝາະສົມສຳລັບການພັດທະນາລະບົບປັນຍາປະດິດ ແລະ Machine Learning , Python ຖືກນຳໃຊ້ເພື່ອການປະມວນຜົນ,ວິເຄາະຂໍ້ມູນການພັດທະນາ AI/ML Model, ເປັນ High-level programming language ທີ່ງ່າຍຕໍ່ການຮຽນ, ມີຊຸດ library ຫຼາຍ ແລະ ໃຊ້ໄດ້ຫຼາຍໃນ Data science, AI, web development.

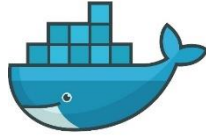
2.4.5 ແນະນຳ Pytorch



ຮູບພາບ 2.6 ສະແດງຮູບພາບ Pytorch

PyTorch ແມ່ນ Framework ສຳລັບ Deep Learningທີ່ນິຍົມໃນວົງການ AI, PyTorch ເປັນ Open-Source Machine Learning Library ທີ່ສຳລັບ Python, ເຊັ່ນ Deep learning (neural networks). Dynamic computation graph ສະດວກໃນການ experiment ແລະ debugging, ມີ GPU acceleration ເພື່ອ training models ໄວ, ມີ community ໃຫຍ່ ແລະ tutorials ຫຼາຍ.

2.4.6 ແນະນຳ Docker



ຮູບພາບ 2.7 ສະແດງຮູບພາບ Docker

Docker ແມ່ນ ເຕັກໂນໂລຢີສຳລັບການຮັບໂປຣແກຣມແບບ Container ທີ່ຊ່ວຍໃຫ້ນັກພັດທະນາສາມາດ “ຫໍ່” (package) ໂປຣແກຣມ + library + dependency + config ໄວ້ນຳກັນ ແລ້ວນຳໄປຮັບໄດ້ທຸກບ່ອນ ໂດຍບໍ່ຂຶ້ນກັບ OS ຫຼື environment.

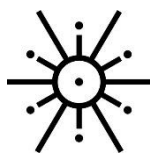
2.4.7 ແນະນຳ Fast API



ຮູບພາບ 2.8 ສະແດງຮູບພາບ Fast API

FastAPI ແມ່ນ Python Web Framework ທີ່ທັນສະໄໝ ແລະ ໄວຫຼາຍ ໃຊ້ສຳລັບສ້າງ API (Backend / REST API) ໂດຍເນັ້ນຄວາມໄວ, ຄວາມປອດໄພ, ແລະ developer experience.

2.4.8 ແນະນຳ Tokio + Axum



ຮູບພາບ 2.9 ສະແດງຮູບພາບ Tokio + Axum

Tokio ແມ່ນ runtime ແບບຕໍ່ເນື່ອງ ແລະ Axum ແມ່ນ Web Framework ຂອງພາສາ Rust ທີ່ໃຊ້ສໍາລັບສ້າງ Backend / API / Web Server ທີ່ໄວ, ປອດໄພ ແລະ ທັນສະໄໝ.

2.4.9 ແນະນຳ NIX



ຮູບພາບ 2.10 ສະແດງຮູບພາບ NIX

Nix ແມ່ນ package manager ແລະ build system ທີ່ອອກແບບມາໃຫ້ການຕິດຕັ້ງໂປຣແກຣມແນ່ນອນ, ຊື້ໄດ້, ແລະ ປອດໄພ ກວ່າ package manager ແບບດັ້ງເດີມ (apt, pacman, dnf ແລະອື່ນໆ).

2.4.10 ແນະນຳ Google Colab



ຮູບພາບ 2.11 ສະແດງຮູບພາບ Google Colab

Google Colab ແມ່ນບໍລິການຂອງ Google ທີ່ໃຫ້ເຮົາຂຽນ ແລະ ຮັບ ຮັບ code Python (Machine Learning / AI, Data Science & Data Analysis) ຜ່ານ browser ໂດຍ ບໍ່ຕ້ອງຕິດຕັ້ງຫຍັງໃນເຄື່ອງ ຄ້າຍຄື Jupyter Notebook.

2.4.11 ແນະນຳ Kaggle



ຮູບພາບ 2.12 'ແນະນຳ Kaggle

Kaggle ແມ່ນເວັບໄຊທ໌ຊຸມຊົນອອນໄລນ໌ສຳລັບນັກວິທະຍາສາດຂໍ້ມູນ ແລະນັກ Machine Learning ທົ່ວໂລກ, ທີ່ໃຫ້ທ່ານເຂົ້າຮ່ວມການແຂ່ງຂັນ, ດາວໂຫຼດ/ແບ່ງປັນ dataset, ຝຶກຮຽນ, ແລະ ສ້າງ portfolio ຂອງຕົນເອງ. ປັດຈຸບັນ Kaggle ເປັນບໍລິສັດລູກຂອງ Google LLC.

2.4.12 ແນະນຳ Hugging Face



ຮູບພາບ 2.13 ແນະນຳ Hugging Face

Hugging Face ແມ່ນແພລດຟອມຊຸມຊົນ AI ທີ່ເປີດແຫຼ່ງໂປແກຼມ (open-source) ໃຫ້ນັກພັດທະນາທົ່ວໂລກແບ່ງປັນ ແລະ ຮ່ວມມືກັນໃນການສ້າງແອັບພລິເຄຊັນປັນຍາປະດິດ (AI). ມັນໂດດເດັ່ນໃນການພັດທະນາດ້ານ Natural Language Processing (NLP), Computer Vision, ສຽງ ແລະ Machine Learning.

2.4.13 ແນະນຳ Apache Kafka



ຮູບພາບ 2.14 ແນະນຳ Apache Kafka

Apache Kafka ແມ່ນແພລດຟອມ *open-source* ສໍາລັບ *event streaming* ແບບກະຈາຍ (distributed) ທີ່ຖືກນໍາໃຊ້ໂດຍບໍລິສັດຂະໜາດໃຫຍ່ທົ່ວໂລກ ເພື່ອຈັດການຂໍ້ມູນຈຳນວນຫຼາຍແບບ *real-time*. ມັນຊ່ວຍໃຫ້ການຮັບ-ສົ່ງຂໍ້ມູນ, ການເກັບຮັກສາ ແລະ ການປະມວນຜົນຂໍ້ມູນເກີດຂຶ້ນຢ່າງມີປະສິດທິພາບ.

ບົດທີ 3

ວິເຄາະ ແລະ ອອກແບບ

3.1 ວິເຄາະ

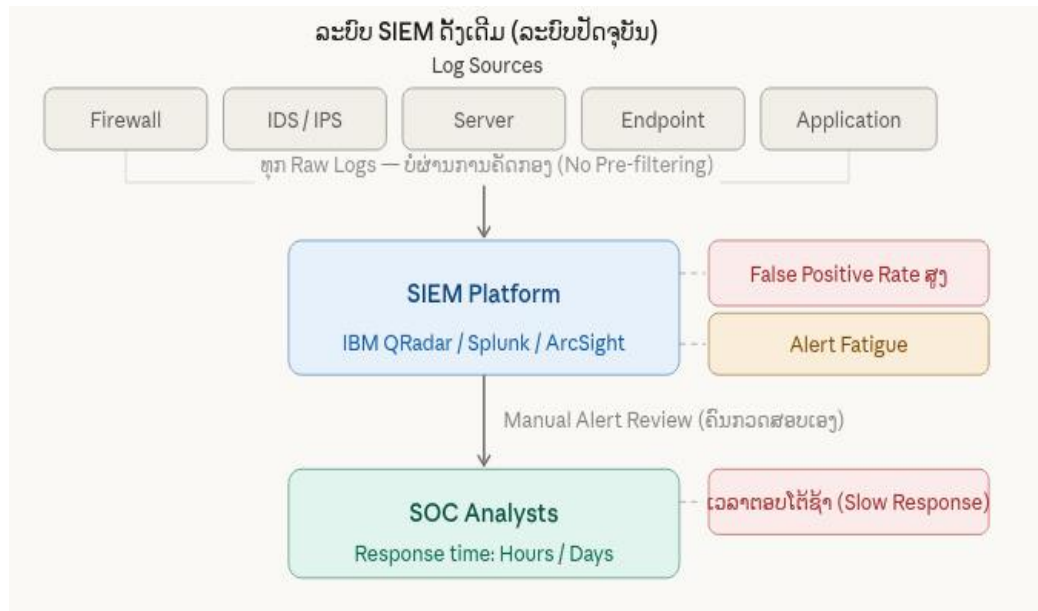
ໃນບົດນີ້ຈະນຳສະເໜີຜົນການວິເຄາະບັນຫາຂອງລະບົບ SIEM ແບບດັ້ງເດີມ, ການກຳນົດຄວາມຕ້ອງການຂອງລະບົບໃໝ່ ແລະ ການອອກແບບສະຖາປັດຕະຍະກຳ (Architecture) ຂອງລະບົບກວດຈັບ ແລະ ຕອບສະໜອງໄພຄຸກຄາມດ້ວຍປັນຍາປະດິດ (AI-Powered Threat Detection and Response System) ເຊິ່ງສອດຄ່ອງກັບຈຸດປະສົງ ແລະ ຂອບເຂດການສຶກສາທີ່ໄດ້ລະບຸໄວ້ໃນບົດທີ 1 ແລະ ອີງໃສ່ທິດສະດີ ແລະ ເຄື່ອງມືທີ່ໄດ້ນຳສະເໜີໃນບົດທີ 2.

3.1.1 ການເຮັດວຽກຂອງລະບົບປັດຈຸບັນ

ໃນສະພາບແວດລ້ອມດ້ານຄວາມປອດໄພທາງໄຊເບີໃນປັດຈຸບັນ, ອົງກອນຕ່າງໆ ໂດຍສະເພາະອົງກອນຂະໜາດໃຫຍ່ໃນ ສປປ ລາວ ເຊັ່ນ: ບໍລິສັດ Lao Telecom (LTC), ທະນາຄານການຄ້າຕ່າງປະເທດລາວ (BCEL) ແລະ ໄຟຟ້າລາວ (EDL) ມັກນຳໃຊ້ລະບົບ SIEM (Security Information and Event Management) ເຊັ່ນ IBM QRadar, Splunk ຫຼື ArcSight ເປັນເຄື່ອງມືຫຼັກໃນການຈັດການ ແລະ ວິເຄາະເຫດການດ້ານຄວາມປອດໄພ. ລະບົບ SIEM ແບບດັ້ງເດີມດຳເນີນງານຕາມ 4 ຂັ້ນຕອນດັ່ງນີ້:

- 1) ການເກັບກຳຂໍ້ມູນ (Data Collection): ຮັບຂໍ້ມູນ Log Events ຈາກແຫຼ່ງຕ່າງໆ ລວມທັງ Firewall, IDS/IPS, Server, Endpoint ແລະ Application ໂດຍໃຊ້ Syslog (RFC 5424), JSON Log, CEF ຫຼື LEEF ເປັນຮູບແບບການສົ່ງຂໍ້ມູນ.
- 2) ການຮວບຮວມເຂົ້າສູ່ສູນກາງ (Centralized Aggregation): ສົ່ງຂໍ້ມູນ Logs ທັງໝົດເຂົ້າສູ່ລະບົບ SIEM ໂດຍກົງ ໂດຍບໍ່ມີການກັ່ນຕອງ ຫຼື ຈັດລຳດັບຄວາມສຳຄັນກ່ອນ.
- 3) ການວິເຄາະດ້ວຍກົດລະບຽບ (Rule-Based Analysis): ລະບົບ SIEM ນຳໃຊ້ Correlation Rules ທີ່ກຳນົດໄວ້ລ່ວງໜ້າ ເພື່ອກວດຈັບເຫດການທີ່ອາດເປັນໄພຄຸກຄາມ.
- 4) ການແຈ້ງເຕືອນ ແລະ ການສືບສວນດ້ວຍຄົນ (Manual Alert Review): ເມື່ອມີການແຈ້ງເຕືອນ ທີ່ມາຈາກ SOC Analysts ຕ້ອງກວດສອບ ແລະ ວິເຄາະແຕ່ລະເຫດການດ້ວຍຕົນເອງ.

ຈາກຂັ້ນຕອນດັ່ງກ່າວ ຈະເຫັນໄດ້ວ່າລະບົບ SIEM ແບບດັ້ງເດີມຮັບຂໍ້ມູນ Logs ຈາກທຸກແຫຼ່ງໂດຍກົງໂດຍບໍ່ຜ່ານການກັ່ນຕອງເຊິ່ງສົ່ງຜົນໃຫ້ລະບົບຕ້ອງປະມວນຜົນຂໍ້ມູນຈຳນວນມະຫາສານແລະທີມງານSOCຕ້ອງກວດກາການແຈ້ງເຕືອນທີ່ມີຈຳນວນສູງດ້ວຍຕົນເອງ.



ຮູບພາບ 3.1 ສະແດງຮູບພາບການເຮັດວຽກລະບົບປັດຈຸບັນ

ຈາກແຜນພາບຂ້າງເທິງ ຈະເຫັນໄດ້ວ່າ ລະບົບ SIEM ແບບດັ້ງເດີມຮັບຂໍ້ມູນ Logs ຈາກທຸກແຫຼ່ງໂດຍກົງ ໂດຍບໍ່ຜ່ານການກັ່ນຕອງ ເຊິ່ງສົ່ງຜົນໃຫ້ລະບົບຕ້ອງປະມວນຜົນຂໍ້ມູນຈຳນວນມະຫາສານ ແລະ ທີມງານ SOC ຕ້ອງກວດກາການແຈ້ງເຕືອນທີ່ມີຈຳນວນສູງດ້ວຍຕົນເອງ.

3.1.2 ບັນຫາ ແລະ ສາເຫດ

ຈາກການສຶກສາ ແລະ ວິເຄາະລະບົບ SIEM ແບບດັ້ງເດີມທີ່ນຳໃຊ້ໃນປັດຈຸບັນ ພົບບັນຫາຫຼັກ 5 ປະການ ດັ່ງຕໍ່ໄປນີ້:

ຕາຕະລາງ 3.1 ສະແດງບັນຫາ ແລະ ສາເຫດ

ລຳດັບ	ບັນຫາ	ສາເຫດ	ຜົນກະທົບ
1	ອັດຕາ False Positive ສູງ	Correlation Rules ແບບ Static ບໍ່ສາມາດປັບຕົວຕາມພຶດຕິກຳທີ່ປຸງແປງ ເຮັດໃຫ້ແຈ້ງເຕືອນຫຼາຍເຫດການທີ່ບໍ່ແມ່ນໄພຄຸກຄາມຈິງ	ທີມ SOC ເສຍເວລາ 70-80% ໃນການກວດສອບ Alert ທີ່ບໍ່ສຳຄັນ (Alert Fatigue)

2	ກວດຈັບໄພຄຸກຄາມໃໝ່ບໍ່ໄດ້	ລະບົບ Rule-Based ສາມາດກວດຈັບໄດ້ສະເພາະໄພຄຸກຄາມທີ່ຮູ້ຈັກແລ້ວ (Known Threats) ບໍ່ສາມາດກວດຈັບ Zero-day ຫຼື APT ໄດ້	ຄວາມສ່ຽງສູງທີ່ຂໍ້ມູນລະອຽດອ່ອນຈະຖືກເຂົ້າເຖິງໂດຍບໍ່ຮູ້ຕົວ
3	ເວລາຕອບສະໜອງຊ້າ	ການກວດສອບ ແລະ ວິເຄາະເຫດການດ້ວຍຄົນຕ້ອງໃຊ້ເວລາຫຼາຍຊົ່ວໂມງ ຫຼື ຫຼາຍວັນ ໃນຂະນະທີ່ Malware ແຜ່ກະຈາຍໄດ້ພາຍໃນນາ	ຄວາມເສຍຫາຍເພີ່ມຂຶ້ນເມື່ອການຕອບໂຕ້ລ່າຊ້າ
4	ຕົ້ນທຶນດຳເນີນງານສູງ	ຄ່າ License SIEM ຄິດໄລ່ຕາມປະລິມານ EPS (Events Per Second), ຄ່າ Storage ສຳລັບເກັບ Logs ແລະ ຄ່າຈ້າງ SOC Analysts	ອົງກອນໃນ ສປປ ລາວ ຫຼາຍແຫ່ງບໍ່ສາມາດຮັບມືກັບຄ່າໃຊ້ຈ່າຍນີ້ໄດ້
5	ຂາດແຄນບຸກຄະລາກອນ	ສປປ ລາວ ຍັງມີຜູ້ຊ່ຽວຊານດ້ານ Cybersecurity ຈຳກັດໂດຍສະເພາະຜູ້ທີ່ມີປະສົບການໃນ Incident Response	ບໍ່ສາມາດປະຕິບັດງານ SOC 24/7 ໄດ້ຢ່າງມີປະສິດທິພາບ

3.1.3 ແນວທາງແກ້ໄຂ

ບົນພື້ນຖານຂອງບັນຫາທີ່ວິເຄາະຂ້າງເທິງ ແລະ ທິດສະດີ AI/ML ທີ່ໄດ້ສຶກສາໃນບົດທີ 2, ແນວທາງການແກ້ໄຂໄດ້ຖືກກຳນົດອອກເປັນ 4 ດ້ານຫຼັກ ດັ່ງນີ້:

1) ການນຳໃຊ້ AI/ML ເພື່ອກັ່ນຕອງ Logs ກ່ອນເຂົ້າ SIEM (Pre-SIEM AI Filtering): ນຳໃຊ້ ONNX Log Classifier ທີ່ເຫຼືອຈາກ HDFS TraceBench Dataset ສຳລັບ Fast Classification ຮ່ວມກັບ SecureBERT 2.0 ທີ່ ດາວໂຫລດຜ່ານ Hugging Face Hub (cisco-ai/SecureBERT2.0-NER) ສຳລັບ Cybersecurity-domain NLP.

2) ການກວດຈັບໄພຄຸກຄາມດ້ວຍ Sigma Rules ແລະ MITRE ATT&CK Mapping: ນຳໃຊ້ Sigma Rules Engine ຮ່ວມກັບ SecureBERT 2.0 BiEncoder + FAISSທີ່ໂຫລດຜ່ານHugging Face Transformers Library ສຳລັບ Semantic ATT&CK Retrieval.

3) ການຕອບໂຕ້ອັດຕະໂນມັດ (Automated Response): ອອກແບບລະບົບໃຫ້ Block IP, ກັກກັນອຸປະກອນ ແລະ ແຈ້ງເຕືອນ SOC ທັນທີ.

4) ການຫຼຸດຕົ້ນທຶນ ແລະ ແກ້ບັນຫາຂາດແຄນບຸກຄະລາກອນ: Ai Filtering ຈະຊ່ວຍຫຼຸດປະລິມານ EPS ທີ່ເຂົ້າ SIEM ລົງ ເຊິ່ງຫຼຸດຄ່າ License, ຄ່າ Storage ແລະ ຄ່າແຮງງານ.

3.1.4 ຄວາມຕ້ອງການຂອງລະບົບໃໝ່

ຄວາມຕ້ອງການຂອງລະບົບໃໝ່ ຖືກແບ່ງອອກເປັນ 2 ປະເພດ ຄື: ຄວາມຕ້ອງການທາງດ້ານການທຳງານ (Functional Requirements) ແລະ ຄວາມຕ້ອງການທີ່ບໍ່ແມ່ນການທຳງານ (Non-Functional Requirements).

1) ຄວາມຕ້ອງການທາງດ້ານການທຳງານ (Functional Requirements)

- ລະບົບຕ້ອງສາມາດຮັບ Logs ຈາກ Firewall, IDS/IPS, Server, Endpoint, Application ຜ່ານ Syslog, JSON, CEF ແລະ LEEF.

- ລະບົບຕ້ອງສາມາດຈຳແນກ Logs (Log Classification) ໂດຍໃຊ້ AI/ML ເພື່ອແຍກ Logs ທີ່ສຳຄັນ (Critical, High) ອອກຈາກ Logs ທີ່ບໍ່ສຳຄັນ (Informational, Low).

- ລະບົບຕ້ອງສາມາດກວດຈັບໄພຄຸກຄາມແບບ Real-time ໂດຍໃຊ້ Sigma Rules (Fast Pattern Matching) ກັບ SecureBERT 2.0 (Hugging Face) + FAISS.

- ລະບົບຕ້ອງສາມາດຕອບໂຕ້ອັດຕະໂນມັດ ເຊັ່ນ: Block IP, ກັກກັນອຸປະກອນ ແລະ ແຈ້ງເຕືອນ SOC Team.

- ລະບົບຕ້ອງຮອງຮັບ API Gateway (FastAPI) ເພື່ອສົ່ງ Logs ຜ່ານ Kafka ເຂົ້າ IBM QRadar ໃນຮູບແບບ LEEF 2.0.

2) ຄວາມຕ້ອງການທີ່ບໍ່ແມ່ນການທຳງານ (Non-Functional Requirements).

- Performance: ລະບົບຕ້ອງປະມວນຜົນ Logs ໃນລະດັບ Millisecond ເພື່ອ Real-time Detection.

- Reliability: Uptime ບໍ່ຕ່ຳກວ່າ 99.9%.

- Security: ສອດຄ່ອງ ISO 27001, PCI-DSS ແລະ Cybersecurity Law of Lao PDR.

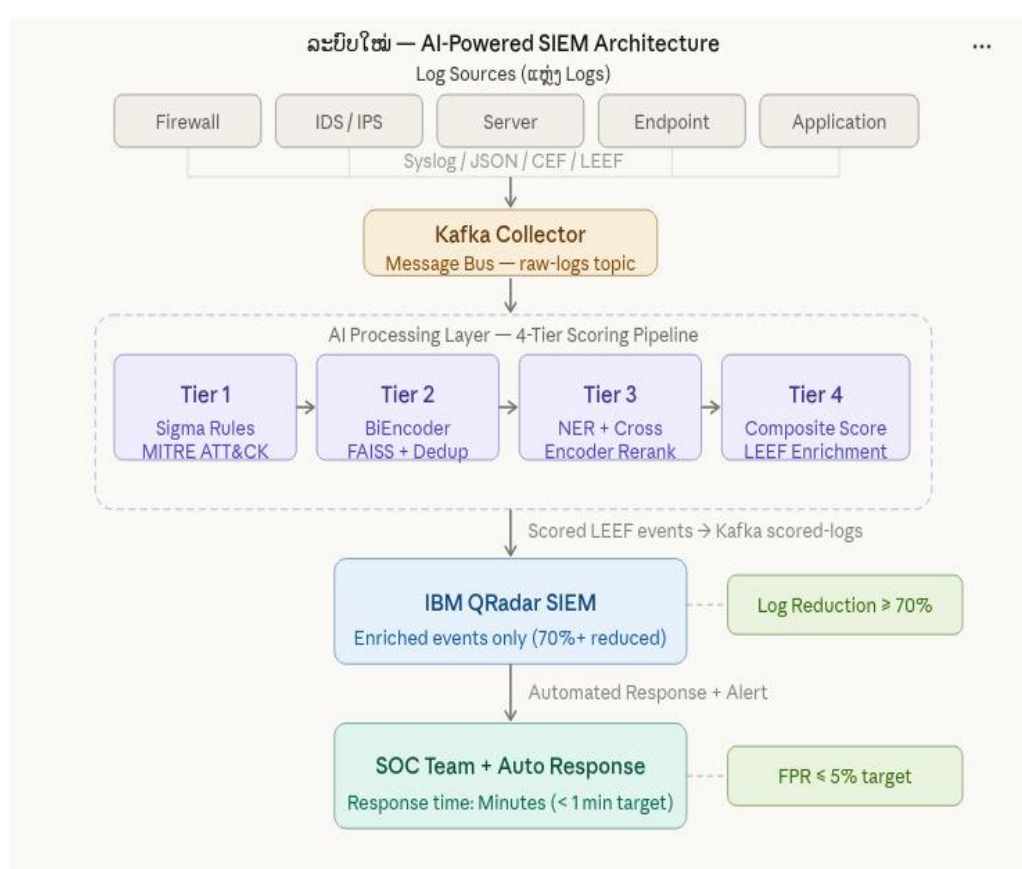
- Scalability: ຮອງຮັບ Logs ທີ່ເພີ່ມຂຶ້ນ ໂດຍໃຊ້ Docker Containerization.

- Usability: Dashboard ເຂົ້າໃຈງ່າຍ ເໝາະກັບ SOC ຂະໜາດນ້ອຍ.

3.2 ການອອກແບບ

3.2.1 ການເຮັດວຽກຂອງລະບົບໃໝ່

ລະບົບໃໝ່ຖືກອອກແບບໃຫ້ມີ AI Filtering Layer ຢູ່ລະຫວ່າງ Log Sources ແລະ IBM QRadar SIEM ເພື່ອກັ່ນຕອງ, ວິເຄາະ ແລະ ຈັດລຳດັບຄວາມສຳຄັນຂອງ Logs ກ່ອນສົ່ງເຂົ້າ SIEM. ສະຖາປັດຕະຍະກຳລະບົບປະກອບດ້ວຍ 3 ຊັ້ນຫຼັກ ຄື: Log Collection Layer, AI Processing Layer (4-Tier Scoring) ແລະ SIEM Forwarding Layer ໂດຍໃຊ້ Kafka ເປັນ Message Bus ກາງ.



ຮູບພາບ 3.2 ສະແດງຮູບພາບການເຮັດວຽກຂອງລະບົບໃໝ່

ຈຸດເດັ່ນທີ່ສຳຄັນທີ່ສຸດຂອງລະບົບໃໝ່ ຄື AI Processing Layer (4-Tier Scoring Pipeline) ເຊິ່ງປະມວນຜົນ Logs ຜ່ານ 4 ຂັ້ນຕອນແບບ Sequential ດັ່ງນີ້:

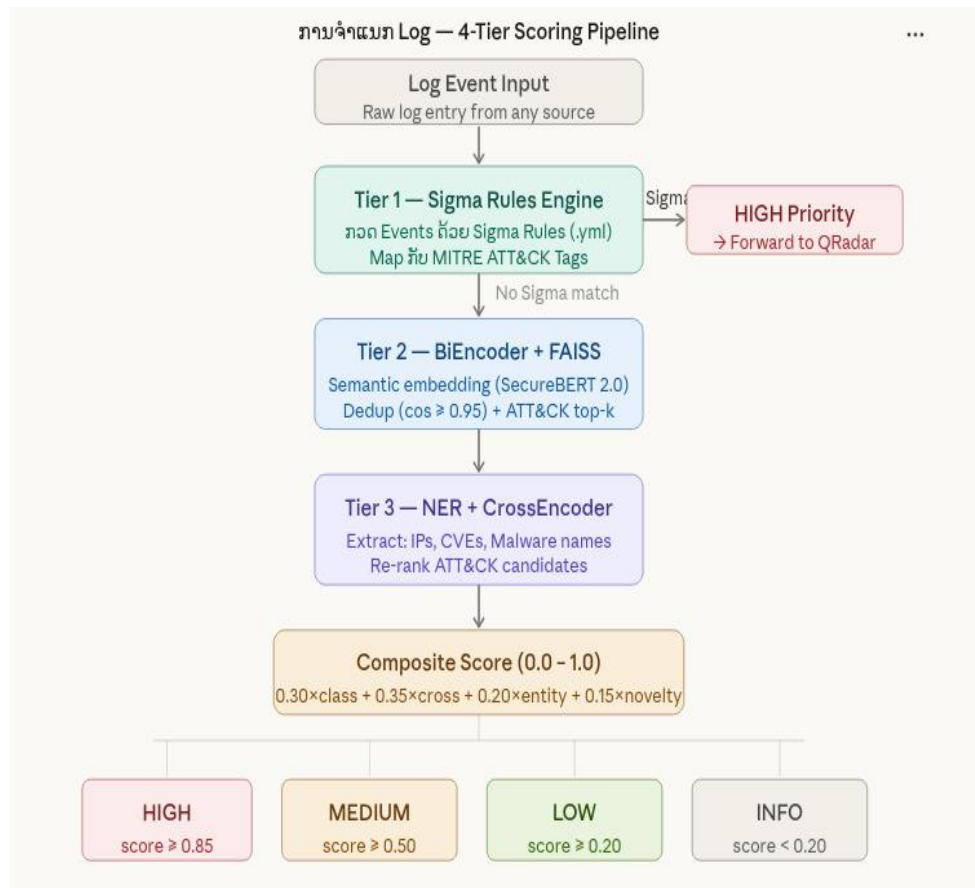
- Tier 1 — Sigma Rule Engine: ກວດ Logs ທຸກ Event ດ້ວຍ Sigma Rules (config/sigma_rules/*.yaml) ທີ່ Map ກັບ MITRE ATT&CK Tags (ເຊັ່ນ: SSH Brute Force T1110, Credential Dump T1003, Lateral Movement T1021). Sigma Match → ຈັດ Priority ເປັນ HIGH ທັນທີ ໂດຍບໍ່ຕ້ອງລໍຖ້າ Tier ຕໍ່ໄປ.

- Tier 2 — BiEncoder Dedup + ATT&CK Retrieval: ໃຊ້ SecureBERT 2.0 BiEncoder (cisco-ai/SecureBERT2.0-biencoder) ສ້າງ Embedding ແລ້ວ Compare ກັບ FAISS Index ສໍາລັບ 2 ຈຸດປະສົງ: (1) Deduplication — ກວດ Duplicate Events (cosine similarity ≥ 0.95 ໃນ Window 5 ນາທີ) ແລະ (2) ATT&CK Candidate Retrieval ຫຼັງຈາກນັ້ນ ດຶງ Top-K MITRE Techniques ທີ່ກ່ຽວຂ້ອງທີ່ສຸດຈາກ Semantic Index.

- Tier 3 — NER + CrossEncoder Re-ranking: ສໍາລັບ Non-Duplicate Events ໃຊ້ SecureBERT 2.0 NER (cisco-ai/SecureBERT2.0-NER) ສະກັດ Entities (IP addresses, CVEs, Malware) ຈາກນັ້ນໃຊ້ CrossEncoder (cisco-ai/SecureBERT2.0-cross_encoder) Re-rank ATT&CK Candidates ເພື່ອເລືອກ Technique ທີ່ Match ທີ່ສຸດ ແລະ ຄໍານວນ Entity Boost Score.

- Final Score Formula: ຄໍານວນ Composite Threat Score (0.0–1.0) ໂດຍ:
$$\text{score} = (0.30 \times \text{classifier_score}) + (0.20 \times \text{entity_boost}) + (0.35 \times \text{cross_encoder_max}) + (0.15 \times \text{novelty_score}) - \text{dedup_penalty}.$$

Routing Thresholds: HIGH ≥ 0.85 , MEDIUM ≥ 0.50 , LOW ≥ 0.20 , INFO < 0.20 .



ຮູບພາບ 3.3 ການຈຳແນກ Log (Log Classification & 4-Tier Scoring)

3.2.2 ຊັບພະຍາກອນທີ່ນຳໃຊ້ພັດທະນາ

ກ. Hardware

1) Laptop ຈຳນວນ 1 ຫນ່ວຍ

- Acer Predator Helios Neo 16 .
- Processor: Intel Core i5 13500H 12.
- GPU: NVIDIA RTX 4050 VRAM 6GB.
- RAM: 32 GB DDR5 Bus 4800 mHz.
- SSD: 500 GB NVMe SSD.

2) Server ຈຳນວນ 1 ເຄື່ອງ

- Server: Ubuntu Server.
- Physical/VM: 24.04 LTS.
- CPU: 6-10cores.
- RAM: 64GB+.
- Storage: 2TB+ SSD.
- Network: 10 Gbps.
- Storage System: NAS/SAN with 10TB+ capacity.

ຂ. Platform ແລະ ເຄື່ອງມື AI (ລວມ Hugging Face)

- Hugging Face Hub: ແຫຼ່ງດາວໂຫລດ Pre-trained Models (SecureBERT 2.0) ສໍາລັບ Project ນີ້.
- Python + PyTorch: ສໍາລັບ Training ແລະ Inference AI/ML Models.
- Hugging Face Transformers Library: ສໍາລັບ Load Models ຈາກ Hugging Face Hub.
- ONNX Runtime: ສໍາລັບ Fast Model Inference.
- FAISS (Facebook AI Similarity Search): ສໍາລັບ Vector Similarity Search ຂອງ ATT&CK Techniques.
- FastAPI (Python): ສໍາລັບ AI Scoring Service API.
- Kafka + Zookeeper: Message Bus ກາງຂອງລະບົບ.
- Docker + Docker Compose: ສໍາລັບ Container Deployment ລະບົບທັງໝົດ.
- Prometheus + Grafana: ສໍາລັບ Monitoring Metrics.
- IBM QRadar: SIEM Platform ຕົ້ນທາງ.

3.3 ການອອກແບບການທົດລອງ

ເພື່ອຕອບຈຸດປະສົງ 3 ຄື ການພິສູດວ່າ SIEM + AI Filtering ມີປະສິດທິພາບດີກວ່າ SIEM ດັ່ງເດີມ, ຈຶ່ງໄດ້ອອກແບບ Controlled Experiment ໂດຍປຸງບທຽບ 2 ສະຖານະການ.

3.3.1 ສະຖານະການທົດລອງ (Experiment Scenarios)

ການທົດລອງໃຊ້ Dataset ຊຸດດຽວກັນ ຕິດຕັ້ງໃນ Environment ດຽວກັນ ແຕ່ຕ່າງກັນທີ່ມີ ຫຼື ບໍ່ມີ AI Filtering Layer ດັ່ງຕາຕະລາງ 3.2: ຕາຕະລາງ 3.2 ສະແດງຄວາມຕ່າງລະຫວ່າງ ສະຖານະການ A (SIEM ດັ່ງເດີມ) ແລະ B (SIEM + AI Filter): Scenario A: Log Sources → IBM QRadar ໂດຍກົງ (ບໍ່ມີ AI Layer); Scenario B: Log Sources → Collector (Kafka) → AI Scoring API (FastAPI + SecureBERT 2.0) → LEEF Enrichment → IBM QRadar, ໂດຍ Prometheus + Grafana ເກັບ Metrics ທັງ 2 ສະຖານະການ.

ຕາຕະລາງ 3.2 ສະຖານະການທົດລອງ

ລາຍການ	Scenario A — Baseline (SIEM ດັ້ງເດີມ)	Scenario B — Proposed (SIEM + AI Filter)
ສະຖາປັດຕະຍະກຳ	Log Sources → SIEM ໂດຍກົງ (ບໍ່ມີ AI Layer)	Log Sources → AI Filtering Layer → SIEM
Dataset	CICIDS2017, NSL-KDD, Log Hub (ຊຸດດຽວກັນ)	CICIDS2017, NSL-KDD, Log Hub (ຊຸດດຽວກັນ)
SIEM Platform	IBM QRadar	IBM QRadar + AI Filtering Layer (Python)
ໄລຍະທົດລອງ	72 ຊົ່ວໂມງ (Simulation + Replay)	72 ຊົ່ວໂມງ (Simulation + Replay)
Hardware	Server: Ubuntu 24.04 LTS, 64GB RAM, 10 Gbps	Server ດຽວກັນ + AI Model ຮັ່ງໃນ Docker
Metrics	Detection Rate, FPR, CPU, Memory, Storage, Response Time, Cost	Detection Rate, FPR, Log Reduction Rate, CPU, Memory, Storage, Response Time, Cost

3.3.2 ຂໍ້ມູນ Dataset ທີ່ໃຊ້ໃນການທົດລອງ

Dataset ທີ່ເລືອກສຳລັບ Train, Validate ແລະ ທົດລອງໃນ 2 ສະຖານະການ ມີ 4 ຊຸດ ດັ່ງຕໍ່ໄປນີ້:

ຕາຕະລາງ 3.3 ຂໍ້ມູນ Dataset

Dataset	ລາຍລະອຽດ	ຈຳນວນ Records	ການໃຊ້ງານ
CICIDS2017	Network Intrusion ຈາກ Univ. of New Brunswick — 15 ປະເພດ Attack	~2.8 ລ້ານ	Train Supervised Model
NSL-KDD	ປັບປຸງຈາກ KDD Cup 99 — 4 ໝວດ Attack: DoS, R2L, U2R, Probe	~125,000	Benchmark ປຽບທຽບ
Log Hub	System Logs ຈາກ Real-world (HDFS, BGL, Windows, Linux)	~77 ລ້ານ	Test Log Parsing & Anomaly
Anonymized Logs	Logs ຈິງທີ່ Anonymize ຈາກ Partner Organizations ໃນ ສປປ ລາວ	TBD	Validate ໃນ Real Context

ຂໍ້ມູນ CICIDS2017 ແລະ NSL-KDD ຈະຖືກນຳໃຊ້ Replay ຜ່ານລະບົບ ໃຫ້ Scenario A ແລະ B ຮັບ Traffic ດຽວກັນ ເພື່ອຮັບປະກັນຄວາມຍຸດຕິທຳຂອງການປຽບທຽບ (Fair Comparison).

3.3.3 Metrics ການປະເມີນ ແລະ ເກນຕັດສິນ

Metrics ທີ່ໃຊ້ວັດຜົນປຽບທຽບ ອ້າງອີງຈາກ Confusion Matrix ທີ່ໄດ້ກ່າວໃນບົດທີ 2 ພ້ອມເກນຕັດສິນ ດັ່ງສະແດງໃນຕາຕະລາງ 3.4

ຕາຕະລາງ 3.4 ການປະເມີນ ແລະ ການຕັດສິນ

Metric	ຄວາມໝາຍ	ເກນຂັ້ນຕໍ່າ	ເປົ້າໝາຍ
Detection Rate (Recall)	ອັດຕາການກວດຈັບ True Incidents ຕ້ອງການສູງ	$\geq 90\%$	$\geq 95\%$
False Positive Rate (FPR)	ອັດຕາ Alert ຜິດຕ້ອງການຕໍ່າ	$\leq 10\%$	$\leq 5\%$
Log Reduction Rate	% Logs ທີ່ AI ກັ່ນຕອງໄດ້ກ່ອນສົ່ງ SIEM	$\geq 50\%$	$\geq 70\%$
Precision	ຄວາມຖືກຕ້ອງຂອງ Positive Prediction	$\geq 85\%$	$\geq 92\%$
F1-Score	ຄ່າ Balance ລະຫວ່າງ Precision ແລະ Recall	$\geq 88\%$	$\geq 93\%$
CPU / Memory Usage	ການໃຊ້ Resources ຂອງ SIEM Platform	Baseline -20%	Baseline -40%
Response Time	ເວລາຕອບໂຕ້ຈາກກວດພົບຈົນດຳເນີນການ	≤ 5 ນາທີ	≤ 1 ນາທີ
Estimated Cost Reduction	ການຄາດຄະເນຄ່າ License + Storage + Man-hour	$\geq 30\%$	$\geq 50\%$

ໝາຍເຫດ: Log Reduction Rate ເປັນ Metric ທີ່ Scenario B ມີເທົ່ານັ້ນ ເພາະ Scenario A ບໍ່ມີ AI Filtering. Detection Rate ຈະຕ້ອງ ຮັກສາ ຫຼື ດີຂຶ້ນເທົ່ານັ້ນ, ບໍ່ຍອມໃຫ້ຫຼຸດລົງ.

3.3.4 ຂັ້ນຕອນການທົດລອງ

ຂັ້ນຕອນການທົດລອງທີ່ຮັບປະກັນຄວາມໜ້າເຊື່ອຖື (Reproducibility) ມີ 5 ຂັ້ນຕອນ ດັ່ງນີ້:

1) Environment (Environment Setup): ຕິດຕັ້ງ Scenario A (SIEM ລ້ວນ) ແລະ Scenario B (SIEM + AI) ໃນ Docker Container ດຽວກັນ ໂດຍໃຊ້ Infrastructure as Code (IaC) ເພື່ອຮັບປະກັນ Environment ດຽວກັນ.

2) Replay Traffic: ໃຊ້ tcpreplay ຫຼື ເຄື່ອງມືທີ່ຄ້າຍຄືກັນ Replay CICIDS2017 + NSL-KDD Traffic ຜ່ານທັງ 2 Scenario ໃນໄລຍະ 72 ຊົ່ວໂມງ ເພື່ອໃຫ້ Input ເທົ່າທຽມກັນ.

3) ເກັບ Metrics: ໃຊ້ Prometheus + Grafana ເກັບ CPU, Memory, Logs per second, Alert count, Response Time ທຸກ 1 ນາທີ ຕະຫຼອດໄລຍະທົດລອງ.

4) ປະເມີນ AI Model: ໃຊ້ Confusion Matrix ຄຳນວນ Recall, FPR, Precision, F1-Score ຈາກ Labeled Ground Truth ຂອງ Dataset. 13) ວິເຄາະ ແລະ ສະຫຼຸບ: ນຳ Metrics ທັງໝົດຂອງ A vs B ມາປຸງປະມວນຊົມດ້ວຍ Statistical Test (Mann-Whitney U Test) ເພື່ອຢືນຢັນວ່າຄວາມຕ່າງທີ່ພົບ ມີນັ້ນສຳຄັນທາງສະຖິຕິ ($p\text{-value} < 0.05$).

3.3.5 ວິທີການພິສູດຜົນ (Proof Method)

ລະບົບຖືກຖືວ່າ ພິສູດສຳເລັດ ກໍຕໍ່ເມື່ອ Scenario B ໄດ້ຜົນ $\geq$ ເກນຂັ້ນຕໍ່າ ໃນທຸກ Metric ຫຼັກ ແລະ ຜ່ານ Statistical Test ດ້ວຍ. ການສະແດງຜົນການພິສູດຈະອອກ 3 ຮູບແບບ:

- ຕາຕະລາງຄ່ານູເມຣິກ: ສະແດງຄ່າ Metric ແຕ່ລະອັນຂອງ Scenario A vs B ທຽບກັບ Baseline.

- ກຣາຟ Time Series: ສະແດງ CPU/Memory ໂລດຕາມເວລາ ຂອງທັງ 2 Scenario.

- Confusion Matrix Heatmap: ສະແດງ TP, FP, TN, FN ຂອງ AI Model ໃນ Scenario B.

3.4 ຂັ້ນຕອນການພັດທະນາລະບົບ

ການພັດທະນາລະບົບ AI-Powered SIEM ແບ່ງອອກເປັນ 5 ຂັ້ນຕອນຫຼັກ ດ້ານລຸ່ມ:

3.4.1 ການກະກຽມ ແລະ ການເກັບກຳ Dataset

ລວບລວມ Logs ຈາກ Firewall, IDS/IPS, Server ແລະ Endpoint ກະກຽມ Dataset ທັງ Labeled ແລະ Unlabeled ສຳລັບ Training ໂດຍໃຊ້ CICIDS2017, NSL-KDD, Log Hub HDFS (HDFS_v3_TraceBench ~77 ລ້ານ Records) ຮ່ວມກັບ Anonymized Logs ຈາກ Partner Organizations ໃນ ສປປ ລາວ.

3.4.2 ການພັດທະນາ AI/ML Models

ພັດທະນາ Models ໂດຍໃຊ້ Python + PyTorch/ONNX Runtime ເທຣນເທິງ GPU ແລະ Export ເປັນ ONNX Format ສຳລັບ Fast Inference. Models ທັງໝົດທີ່ອີງໃສ່ SecureBERT 2.0 ຈະຖືກໂຫລດຜ່ານ Hugging Face Transformers Library ຕາມທີ່ໄດ້ອະທິບາຍປະກອບດ້ວຍ:

- Log Classifier: ONNX Model (models/log_classifier.onnx) ເທຣນຈາກ HDFS TraceBench Dataset ໃຊ້ Threshold 0.6 ຂອງ Binary Classification (Threat / Normal), Inference ດ້ວຍ ONNX Runtime.

- SecureBERT 2.0 NER Model (cisco-ai/SecureBERT2.0-NER): Cybersecurity-domain NER ສະກັດ IP addresses, CVEs, Malware, min_confidence 0.80; batch_size 32.

- SecureBERT 2.0 BiEncoder + CrossEncoder: (cisco-ai/SecureBERT2.0-biencoder) + FAISS Index, Dedup (threshold 0.95) ແລະ ATT&CK Retrieval (top-k=3); (cisco-ai/SecureBERT2.0-cross_encoder) Re-rank Candidates ດ້ວຍ Cross-attention.

3.4.3 ການນຳໃຊ້ Hugging Face ເຂົ້າໃນ Project

ຕາມຄຳແນະນຳຂອງອາຈານທີ່ປຶກສາ ທົ່ວຂໍ້ນີ້ຈະອະທິບາຍຢ່າງລະອຽດວ່າ Project ນີ້ນຳໃຊ້ Hugging Face ໃນລັກສະນະໃດ ໃນ 4 ດ້ານຫຼັກ ດັ່ງນີ້:

1) Hugging Face Hub ແຫຼ່ງດາວໂຫຼດ Pre-trained Models

Hugging Face Hub (huggingface.co) ແມ່ນ Platform ທີ່ Project ນີ້ໃຊ້ເພື່ອດາວໂຫຼດ Pre-trained Models ໂດຍກົງ. ໃນ Project ນີ້ ຈະໃຊ້ Models ຈາກ Organization cisco-ai ໃນ Hugging Face ທັງໝົດ 3 Models ຄື:

ຕາຕະລາງ 3.5 Hugging Face Hub

Model ID (Hugging Face Hub)	ປະເພດ	ໜ້າທີ່ໃນ Project
cisco-ai/SecureBERT2.0-NER	Named Entity Recognition	ສະກັດ IP, CVEs, Malware, Organizations ຈາກ Log text (Tier 3)
cisco-ai/SecureBERT2.0-biencoder	Bi-Encoder Embedding	ສ້າງ Vector Embeddings + FAISS Index ສຳລັບ Dedup ແລະ ATT&CK Retrieval (Tier 2)
cisco-ai/SecureBERT2.0-cross_encoder	Cross-Encoder Re-ranking	Re-rank ATT&CK Candidates ເພື່ອຄັດເລືອກ

		Technique ທີ່ Match ທີ່ສຸດ (Tier 3)
--	--	--

2) Hugging Face Transformers Library ດ້ວຍວິທີ Load Models ໃນ Code

ໃນ Code ຂອງ Project ນີ້ ໃຊ້ Hugging Face Transformers Library ໂດຍ Install ຜ່ານ pip ແລ້ວ Import ໃນ Python. ຂັ້ນຕອນການ Load Models ມີ 2 ຂັ້ນຕອນຄື:

- ຕິດຕັ້ງ Library: pip install transformers torch sentencepiece ແລະ pip install huggingface_hub
- ໂຫລດ Model ໃນ Code ໂດຍ Model ຈະຖືກດາວໂຫລດ ແລະ Cache ໄວ້ Local ໃນຄັ້ງທຳອິດ ຈາກນັ້ນ Load ຈາກ Cache ໃນການໃຊ້ຄັ້ງຕໍ່ໄປ:

ຕົວຢ່າງ Code ສຳລັບ SecureBERT 2.0 NER (Tier 3):

```

1 from transformers import AutoTokenizer, AutoModelForTokenClassification, pipeline
2 import torch
3 MODEL_NAME = "cisco-ai/SecureBERT2.0-NER"
4 tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
5 model = AutoModelForTokenClassification.from_pretrained(MODEL_NAME)
6 ner_pipeline = pipeline("ner", model=model, tokenizer=tokenizer,
7                          device=-1 if torch.cuda.is_available() else -1)
8

```

ຮູບພາບ 3.4 ຕົວຢ່າງ code(Tier2)

ຕົວຢ່າງ Code ສຳລັບ SecureBERT 2.0 BiEncoder (Tier 2):

```

1 from sentence_transformers import SentenceTransformer
2 import faiss, numpy as np
3 biencoder = SentenceTransformer("cisco-ai/SecureBERT2.0-biencoder")
4 embedding = biencoder.encode(log_text, normalize_embeddings=True)
5

```

ຮູບພາບ 3.5 ຕົວຢ່າງ code(Tier3)

3) Hugging Face Hub CLI ການຈັດການ Model Cache ໃນ Docker

ເນື່ອງຈາກລະບົບ Deploy ໃນ Docker Container ຈຶ່ງໃຊ້ huggingface-cli ເພື່ອ Pre-download Models ໄວ້ກ່ອນ Build Image ເພື່ອຫຼຸດ Startup Time:

huggingface-cli

download cisco-ai/SecureBERT2.0-NER

- --cache-dir /app/models/hf_cache

huggingface-cli download cisco-ai/SecureBERT2.0-biencoder \

- --cache-dir /app/models/hf_cache

huggingface-cli download cisco-ai/SecureBERT2.0-cross_encoder \

- --cache-dir /app/models/hf_cache

Set Environment Variable: TRANSFORMERS_CACHE=/app/models/hf_cache ໃນ docker-compose.yml ເພື່ອໃຫ້ Transformers Library ອ່ານ Models ຈາກ Volume ທີ່ Pre-download ໄວ້.

4) ເຫດຜົນທີ່ເລືອກໃຊ້ Hugging Face ໃນ Project ນີ້

ຕາຕະລາງ 3.6 ເຫດຜົນຂອງການນຳໃຊ້ Hugging Face

ຂໍ້ດີ	ການນຳໃຊ້ໃນ Project ນີ້
Open Source & ຟຣີ	ດາວໂຫລດ SecureBERT 2.0 ໂດຍບໍ່ຕ້ອງຊື້ License
Domain-specific Model	SecureBERT 2.0 ຖືກ Fine-tune ສຳລັບ Cybersecurity NLP ໂດຍສະເພາະ ເພາະກວ່າ BERT ທົ່ວໄປ
ງ່າຍໃນການ Integrate	ໃຊ້ AutoTokenizer.from_pretrained() ແລະ pipeline() ໂຫລດ Model ໄດ້ທັນທີ
Reproducibility	Model Version ຄົງທີ່ ທົດລອງຊ້ຳໄດ້ຜົນດຽວກັນ ສຳລັບ Scientific Comparison
Docker-friendly	Pre-download ດ້ວຍ huggingface-cli ເຂົ້າ Volume ລະບົບ Boot ໄວ ບໍ່ຕ້ອງ Download ຊ້ຳ

3.4.4 ການພັດທະນາ Backend System

ພັດທະນາ AI Scoring Service ດ້ວຍ Python FastAPI ສຳລັບ Real-time Log Scoring ຜ່ານ Endpoints: POST /score (Single Event), POST /score/batch (ສູງສຸດ 200 Events), GET /health, GET /metrics (Prometheus). Log Collector Service ຮັບ Syslog/HTTP ແລ້ວ

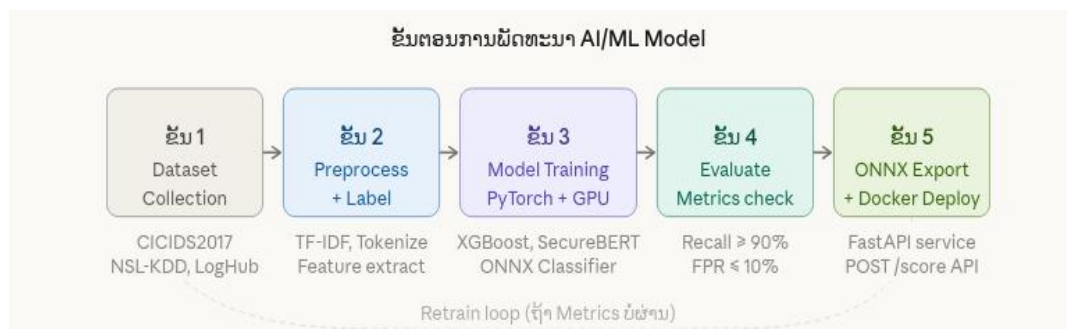
Publish ເຂົ້າ Kafka Topic 'raw-logs'; Router Service Consume ຈາກ Kafka, ໃຊ້ LEEF Enricher Enrich ດ້ວຍ AI Scores ແລ້ວ Forward ເຂົ້າ IBM QRadar ຜ່ານ Syslog.

3.4.5 ການລວມລະບົບ (System Integration)

ເຊື່ອມຕໍ່ AI Filtering Layer ກັບ IBM QRadar SIEM ຜ່ານ LEEF 2.0 Format Syslog Forwarding. Flow ຂອງລະບົບ: Log Sources → Collector (Syslog UDP/TCP) → Kafka 'raw-logs' Topic → LogFilter API (AI Scoring + Hugging Face Models + MITRE Mapping) → Router (LEEF Enrichment) → Kafka 'scored-logs' → QRadar Syslog Receiver. ທົດສອບ Integration ດ້ວຍ pytest (Unit + Integration Tests) ແລະ Smoke Test ກ່ອນ Production.

3.4.6 ການທົດສອບ ແລະ ການປະເມີນ

ທົດສອບລະບົບດ້ວຍ Metrics ທີ່ກຳນົດໃນ 3.3.3 ລວມທັງ Detection Rate, False Positive Rate, Log Reduction Rate ແລະ Response Time ພ້ອມໃຊ້ Confusion Matrix ປະເມີນ AI Model Quality.



ຮູບພາບ 3.6 ຂັ້ນຕອນການພັດທະນາ AI/ML

3.5 ການຈັດການຄຸນນະພາບ

3.5.1 ການຮັບປະກັນຄຸນນະພາບ AI/ML Models

ເກນຮັບປະກັນຄຸນນະພາບ ດັ່ງສະແດງໃນຕາຕະລາງ 3.5:

ຕາຕະລາງ 3.7 ການຮັບປະກັນຄຸນນະພາບ AI/ML

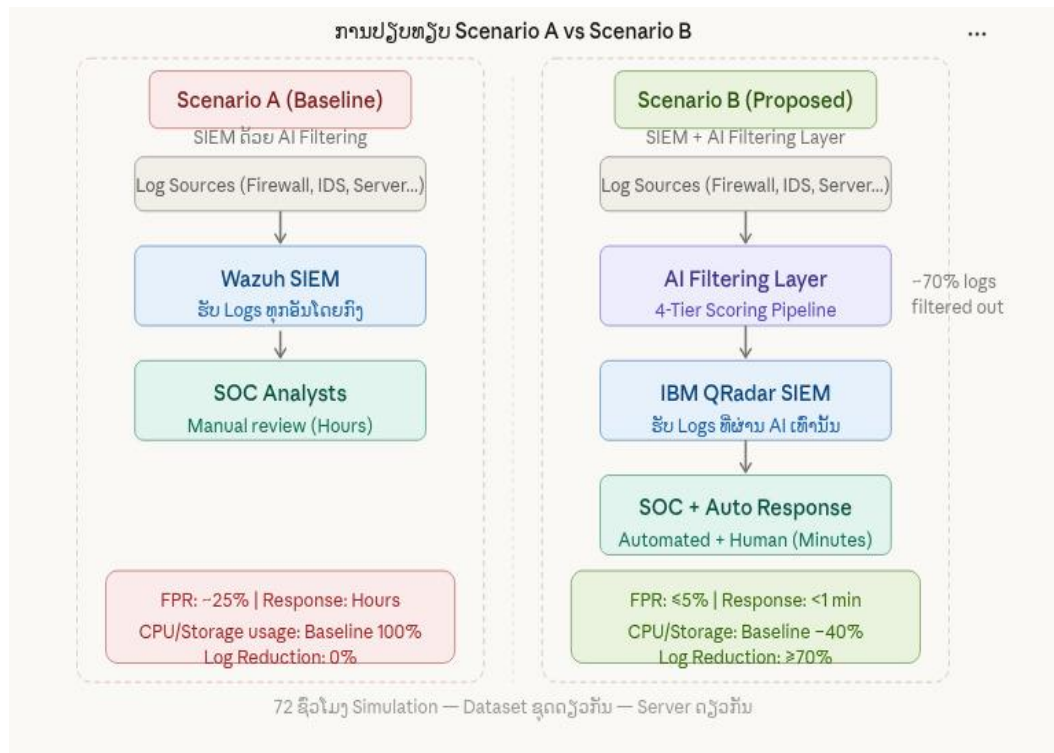
Metric	ເກນຂັ້ນຕໍ່າ	ເປົ້າໝາຍ
Detection Rate (Recall)	≥ 90%	≥ 95%
False Positive Rate	≤ 10%	≤ 5%
Log Reduction Rate	≥ 50%	≥ 70%
Precision	≥ 85%	≥ 92%
F1-Score	≥ 88%	≥ 93%

3.5.2 ການທົດສອບລະບົບ (System Testing)

- Unit Testing: ທົດສອບແຕ່ລະ Component (Hugging Face Models, ONNX, API, Backend).
- Integration Testing: ທົດສອບການເຊື່ອມຕໍ່ທຸກ Components ລວມ Hugging Face Pipeline.
- Performance Testing: ທົດສອບ Throughput, Latency ແລະ GPU Memory Usage ຂອງ Hugging Face Models ພາຍໃຕ້ Load ຕ່າງໆ.
- Security Testing: ທົດສອບ API ແລະ Compliance ຕາມ ISO 27001.

3.5.3 ການປະເມີນປຽບທຽບ (Comparative Evaluation)

ປຽບທຽບ Scenario A (SIEM ດັ້ງເດີມ) ກັບ Scenario B (SIEM + AI Filter ດ້ວຍ Hugging Face) ວັດຈາກ: ປະລິມານ Logs ທີ່ເຂົ້າ SIEM, FPR, Response Time ແລະ ຄ່າໃຊ້ຈ່າຍ.



ຮູບພາບ 3.7 ການປຽບທຽບ Scenario A vs Scenario B

3.6 ການບໍລິຫານຈັດການຄວາມສ່ຽງ

ຕາຕະລາງ 3.8 ການຈັດການຄວາມສ່ຽງ

ລຳດັບ	ຄວາມສ່ຽງ	ລະດັບ	ຜົນກະທົບ	ມາດຕະການຮັບມື
1	Dataset ບໍ່ພຽງພໍ / ບໍ່ມີຄຸນນະພາບ	ສູງ	Model Accuracy ຕ່ຳ	ໃຊ້ CICIDS2017, NSL-KDD ຮ່ວມກັບ Data Augmentation ແລະ Synthetic Data
2	Model Overfitting	ປານກາງ	ຜິດພາດໃນ Production	ໃຊ້ Cross-Validation, Regularization ແລະ Ensemble Methods
3	Real-time Processing ຊ້າ	ສູງ	Logs ບໍ່ທັນເວລາ	ໃຊ້ Rust (Tokio) ສຳລັບ High-Performance Processing ແລະ Optimize Model Inference
4	ເຊື່ອມຕໍ່ SIEM ລົ້ມເຫຼວ	ປານກາງ	Logs ຖືກສູນເສຍ	ອອກແບບ Buffer Queue ແລະ Retry Mechanism ພ້ອມ Fallback Mode
5	ຄວາມສ່ຽງດ້ານຄວາມປອດໄພຂອງ API	ສູງ	ລະບົບຖືກໂຈມຕີ	ໃຊ້ API Authentication, Rate Limiting, Input Validation ແລະ Encryption (TLS)

ການບໍລິຫານຄວາມສ່ຽງຈະຖືກທົບທວນ ແລະ ປັບປຸງຕະຫຼອດໄລຍະເວລາພັດທະນາ ໂດຍອີງໃສ່ກອບ NIST Cybersecurity Framework ເພື່ອຮັບປະກັນວ່າລະບົບມີຄວາມປອດໄພ ແລະ ສອດຄ່ອງກັບ ISO 27001, PCI-DSS ແລະ Cybersecurity Law of Lao PDR.

